

Gustavo Trivelatto Gabriel
Bruno Borges Paschoalinoto

**Laboratory Energy Insight Assistant - An
LLM-leveraged tool for energy usage monitoring
in university laboratories**

São Paulo, SP

2025

Gustavo Trivelatto Gabriel
Bruno Borges Paschoalinoto

Laboratory Energy Insight Assistant - An LLM-leveraged tool for energy usage monitoring in university laboratories

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Com-
putação e Sistemas Digitais da Escola Politéc-
nica da Universidade de São Paulo para
obtenção do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Supervisor: Prof. Dr. Jorge Luis Risco Becerra

São Paulo, SP

2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catálogo-na-publicação

Trivelatto, Gustavo

Laboratory Energy Insight Assistant - An LLM-leveraged tool for energy usage monitoring in university laboratories / G. Trivelatto, B. Paschoalinoto -- São Paulo, 2025.

65 p.

Escola Politécnica da Universidade de São Paulo. Departamento de Engenharia de Computação e Sistemas Digitais.

1.LINGUAGEM NATURAL 2.REDES NEURAIS 3.AGENTES INTELIGENTES I.Universidade de São Paulo. Escola Politécnica. Departamento de Engenharia de Computação e Sistemas Digitais II.t. III.Paschoalinoto, Bruno

*Let us strive for the impossible. Remember the great
achievements throughout history have been the
conquest of what seemed the impossible.
(Charles Chaplin)*

Acknowledgements

Agradecemos, primeiramente, ao Prof. Dr. Jorge Risco, orientador deste projeto, que nos apoiou com ideias e sugestões ao longo de todo o desenvolvimento e que nos forneceu acesso a todas as ferramentas necessárias para dar vida ao nosso trabalho.

Agradecemos também aos nossos pais, que nos apoiaram, torceram por nós e sempre estiveram ao nosso lado quando precisamos, impulsionando-nos a alcançar alturas cada vez maiores. Sem eles, não seríamos metade do que somos hoje.

Por fim, agradecemos a todos os nossos amigos e colegas, com quem compartilhamos risadas e lágrimas, vitórias e derrotas. Sem eles, a jornada universitária teria sido muito mais difícil.

Abstract

Recent advances in large language models (LLMs) have enabled a broad range of applications beyond traditional chatbots, opening opportunities for developing sophisticated, domain-specific tools. In this context, we developed the Laboratory Energy Insight Assistant (LEIA), an LLM-based system designed to simplify the process of monitoring and interpreting energy usage in university laboratories. LEIA builds upon the Energy System Insight Tool (ESIT), originally created by Gustavo Trivelatto during his Master's program in Germany. While ESIT aimed to help users explore energy-system scenarios and understand the effects of model modifications without requiring prior modeling expertise, LEIA adapts this foundation to a new use case focused on energy-usage monitoring.

LEIA is implemented using OpenAI's open-weight GPT-OSS models and LangChain, a Python framework that facilitates the integration of language models into custom applications. Its architecture is organized as an execution graph centered on a primary LLM node, supported by tools that retrieve usage data, generate insights, create visualizations, and access external information via web search.

Evaluation results show that LEIA performs well across most tested scenarios, particularly in producing meaningful insights from the available energy-usage data. Nonetheless, development was hindered by the inherent non-determinism of LLMs and by occasional limitations in accessing cloud-based services required by the system.

Keywords: llm. monitoring. energy consumption. chatbot.

Resumo

Os avanços recentes em modelos de linguagem de grande escala (LLMs) possibilitaram uma ampla variedade de aplicações além dos chatbots tradicionais, abrindo oportunidades para o desenvolvimento de ferramentas sofisticadas e específicas de domínio. Nesse contexto, desenvolvemos o Laboratory Energy Insight Assistant (LEIA), um sistema baseado em LLM projetado para simplificar o processo de monitoramento e interpretação do consumo de energia em laboratórios universitários. O LEIA é baseado no Energy System Insight Tool (ESIT), originalmente criado por Gustavo Trivelatto durante seu programa de Mestrado na Alemanha. Enquanto o ESIT tinha como objetivo ajudar usuários a explorar cenários de sistemas energéticos e entender os efeitos de modificações no modelo sem exigir conhecimento prévio em modelagem, o LEIA adapta essa base a um novo caso de uso focado em monitoramento de consumo energético.

O LEIA foi implementado utilizando os modelos de código aberto GPT-OSS da OpenAI e o LangChain, um framework em Python que facilita a integração de modelos de linguagem em aplicações personalizadas. Sua arquitetura é organizada como um grafo de execução centrado em um nó principal de LLM, apoiado por ferramentas que recuperam dados de uso, geram insights, criam visualizações e acessam informações externas por meio de buscas na web.

Os resultados de avaliação mostram que o LEIA apresenta um bom desempenho na maioria dos cenários testados, especialmente na geração de insights relevantes a partir dos dados disponíveis de consumo energético. No entanto, o desenvolvimento foi dificultado pelo caráter não determinístico dos LLMs e por limitações ocasionais no acesso aos serviços em nuvem necessários para o funcionamento do sistema.

Palavras-chave: llm. monitoramento. consumo de energia. chatbot.

List of Figures

Figure 1 – Example chains that can be built using LangChain.	23
Figure 2 – Representation of the measurement data flow.	32
Figure 3 – Initial version of the graph.	35
Figure 4 – Initial version of the final graph.	36
Figure 5 – Final version of the graph.	37
Figure 6 – LEIA’s graph simplified by using GPT OSS.	39

List of Tables

Table 1 – Performance of finetuned Llama 3 models on key benchmark evaluations. The table compares the performance of the 8B, 70B, and 405B versions of Llama 3 with that of competing models. We boldface the best-performing model in each of three model-size equivalence classes. [△] Results obtained using 5-shot prompting (no CoT). [¶] Results obtained without CoT. [◇] Results obtained using zero-shot prompting. <i>Adapted from The Llama 3 Herd of Models, by Llama Team, AI @ Meta, 2024 (DUBEY et al., 2024)</i>	19
Table 2 – Evaluation of GPT OSS 120B and 20B using low reasoning effort across multiple benchmarks <i>Adapted from gpt-oss-120b & gpt-oss-20b Model Card, by OpenAi Team, 2025 (AGARWAL et al., 2025)</i>	21
Table 3 – Current limitations of model usage in Groq’s free plan	25
Table 4 – Current usage prices for Groq’s models	47
Table 5 – Set of test cases developed to assess the general performance of LEIA	51
Table 6 – Experiment results for the simple use cases	52
Table 7 – Experiment results for the complex use cases	52
Table 8 – Experiment results for the multi-modality use cases	52
Table 9 – Set of test cases developed to assess consistency of the workflow execution for the same input	53
Table 10 – Experiment results for the consistency tests	53

List of abbreviations and acronyms

LEIA	Laboratory Energy Insight Assistant
ESIT	Energy System Insight Tool
GUI	Graphical User Interface
TUI	Text-based User Interface
LLM	Large Language Model
API	Application Programming Interface
UI	User Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit
RAM	Random Access Memory
CUDA	Compute Unified Device Architecture
MoE	Mixture-of-Experts
CLI	Command-line Interface
RAG	Retrieval Augmented Generation
LPU	Language Processing Unit
AI	Artificial Intelligence
GB	Giga Byte
CoT	Chain-of-Thought
ReAct	Reasoning and Acting
E-P	Explain-then-Predict
P-E	Predict-then-Explain
JSON	JavaScript Object Notation
CT	Current Transformer

A/D	analog-to-digital
ARM	Advanced RISC Machine
IPC	Industrial PC
PLC	Programmable Logic Controller
ST	Structured Text
IEC	International Electrotechnical Commission
MQTT	Message Queuing Telemetry Transport
STT	Speech-to-Text
IoT	Internet of Things
URL	Uniform Resource Locator
ID	Identity
SQL	Structured Query Language
CESM	Compact Energy System Modeling Tool
AC	Air Conditioning
TPM	Tokens Per Minute
TPD	Tokens Per Day
RPM	Requests Per Minute
RPD	Requests Per Day

List of symbols

B	Billion
T	Trillion
\$	Dollar sign

Contents

1	INTRODUCTION	14
2	OBJECTIVES	16
3	THEORY	17
3.1	Large Language Models (LLMs)	17
3.2	Architecture Evolution: From Llama 3 to GPT-OSS	20
3.3	Text-based User Interface	21
3.4	Consumption Data Gathering	22
3.5	LangChain	23
3.6	GroqCloud	24
3.7	Tavily	24
3.8	Retrieval Augmented Generation (RAG)	25
3.9	Prompting strategies	26
4	IMPLEMENTATION	29
4.1	Devices and Protocol	30
4.2	Data Gathering	30
4.3	Data Availability	32
4.4	Agent development	33
4.4.1	Initial idea: multi-tool with LangChain	33
4.4.2	Final idea for ESIT: node tree with LangGraph	34
4.4.3	LEIA improvements and final form	37
4.4.4	Tools	38
4.4.4.1	Web Search	39
4.4.4.2	Data Access	40
4.4.4.3	Maintenance History Access	40
4.4.5	Prompt engineering	40
4.5	State	43
4.5.1	Chat history	43
4.5.2	Messenger Integration	45
4.6	Modularity	45
4.7	Pricing	46
4.7.1	Groq	46
4.7.2	LlamaCloud	47
4.7.3	Qdrant	47

4.7.4	Tavily	47
5	EXPERIMENTS	49
5.1	Defining the test cases	49
5.2	Test case results	51
5.3	Consistency tests	52
5.4	Test Results	54
5.5	Tool Performance	54
5.6	Observed problems	55
6	RESULTS	56
6.1	Impacts of non-determinism	56
6.2	Trial and error	56
6.3	Positive results	57
7	OUTLOOK	58
7.1	Future work	58
7.2	Conclusion	59
	BIBLIOGRAPHY	60
	APPENDIX A – MULTITOOL AGENT PROMPT	64

1 Introduction

The management of energy consumption and monitoring within smaller technological environments — such as laboratories and research facilities — has become increasingly relevant in the context of modern engineering education and applied research. In these environments, a stable and efficient energy supply is essential for ensuring the continuous operation of computational systems, instrumentation, and automation equipment. As laboratories evolve to include more complex digital infrastructures, the ability to analyze and optimize their energy usage becomes a key component for sustainability and operational reliability (NREL, 2008).

The growth of automation systems has made it possible to measure and control energy consumption with high precision. Companies such as Beckhoff have developed industrial solutions capable of capturing detailed information about voltage, current, and power consumption across multiple devices and workstations (KG, 2024). However, despite the high degree of technical expertise of laboratory coordinators — often professors and engineers with strong backgrounds in computing and energy systems — the challenge lies not in understanding the data, but in efficiently managing, contextualizing, and extracting insights from the continuous flow of information these systems generate.

Visualizing and correlating energy data with laboratory activities, identifying irregularities, or tracing causes of consumption peaks can be time-consuming tasks, especially when multiple data sources, time periods, and maintenance records must be considered simultaneously. Traditional tools for this purpose often rely on static dashboards or highly specialized GUIs (VERA-PIAZZINI; SCARPA; PERON, 2022), which, while powerful, require manual data exploration and offer limited automation of analytical insights. Thus, even for qualified users, the process of maintaining situational awareness over the laboratory’s energy performance can become cumbersome and inefficient (AFZAL et al., 2022).

In this context, Text-based User Interfaces (TUIs) provide an alternative that emphasizes accessibility and efficiency in human–system interaction (RIVADENEIRA; BEDERSON, 2003). Combined with recent advances in Large Language Models (LLMs) (SHANAHAN, 2024), such interfaces enable natural language communication between users and computational systems. Through LLM-based assistants, users can perform queries, receive automatic summaries, and obtain contextual explanations directly through text-based dialogue, without the need to manually navigate dashboards or construct complex filters.

Some of the most known LLMs in the market today include not only widely

recognized proprietary models such as GPT-4 (ACHIAM et al., 2023), Gemini (TEAM et al., 2023) and Claude (ANTHROPIC, 2023) (developed by OpenAi, Google and Anthropic respectively), but also highly capable open source alternatives, such as Llama3 (DUBEY et al., 2024) and Bloom (SCAO et al., 2023), the first developed by Meta and the second by a group of volunteer researchers and employees from Hugging Face, one of the most used hubs for AI models and datasets hosting in the world (JAIN, 2022). Frameworks such as LangGraph and LangChain (MAVROUDIS, 2024) have emerged as effective tools for designing multi-agent conversational architectures capable of integrating LLMs with structured data and external APIs, creating intelligent and interactive environments.

This work presents the *Laboratory Energy Insight Assistant* (LEIA), a chatbot developed to support professors and managers in overseeing a software engineering laboratory's energy consumption. The assistant integrates with Beckhoff monitoring devices to collect energy usage information, generate analytical insights, and notify users about abnormal patterns such as energy peaks or equipment irregularities. Unlike traditional graphical interfaces, LEIA operates through the Telegram messaging platform, providing a conversational and accessible means of interaction for the user (DELFIANTI et al., 2025).

By leveraging open LLMs and modular integration frameworks, LEIA enables laboratory managers to interact with real-time energy data through natural language commands, facilitating quick decision-making and improving the understanding of consumption patterns. The system also provides access to historical maintenance data and contextual explanations, helping bridge the gap between energy data and actionable laboratory management insights.

This report is organized into six chapters, excluding this introduction. We start by providing a more in depth explanation of the problem we are tackling and the proposed solution in Chapter 2, in Chapter 3, we present the theoretical background of this work, including energy monitoring systems, industrial communication, and LLM-based conversational architectures. The implementation details are discussed in Chapter 4, followed by the experimental procedures in Chapter 5. The analysis of the obtained results is presented in Chapter 6, and finally, the conclusions and future perspectives are outlined in Chapter 7.

2 Objectives

The primary objective of this work is to design, implement, and validate the Laboratory Energy Insight Assistant (LEIA), an intelligent conversational agent aimed at simplifying access to energy consumption data within university laboratories. While modern sensing infrastructures, such as Beckhoff measurement devices, provide granular and high-frequency data, the complexity of extracting meaningful information from these systems often hinders quick decision-making by laboratory managers. Therefore, this project aims to bridge the gap between raw technical data and end-users by leveraging the reasoning capabilities of Large Language Models (LLMs) to interpret natural language queries and provide actionable insights in real-time, effectively functioning as a virtual facility manager.

Specifically, the project seeks to develop a robust agentic architecture capable of orchestrating distinct tools to fulfill complex user requests. A central technical goal is the integration of a widely accessible chat interface, Telegram, with a backend system that utilizes SQL querying, web search and spreadsheets manipulation capabilities. This allows the agent to autonomously navigate between structured data, such as historical energy consumption records and equipment maintenance logs, while also being able to search for up to date information from the web. By doing so, the system is designed to not only answer questions but also perform actions, such as generating visualization plots and diagnosing anomalies, thereby reducing the cognitive load required to monitor laboratory assets.

Furthermore, this study aims to evaluate the viability of using open-weights models and commercially available LLM providers to build responsive and cost-effective AI agents. The objective is to demonstrate that a properly architected system using frameworks like LangGraph can achieve high reliability in tool selection and reasoning tasks, offering a competitive alternative to proprietary models. Through a series of defined experiments, ranging from simple data retrieval to complex multi-step reasoning scenarios, this work intends to assess the agent's accuracy, latency, and ability to mitigate hallucinations, establishing a framework for future applications of LLMs in the context of IoT data analysis and smart campus management.

3 Theory

In this chapter, we will delve into the details and the theoretical background necessary to understand the development of the proposed intelligent assistant. The primary goal of this work is to create a chatbot, integrated via Telegram, to assist a professor in managing a software engineering laboratory. The bot is responsible for collecting energy consumption data using Beckhoff devices, providing queries and insights on this data, accessing maintenance histories, and notifying the user of irregularities such as energy spikes.

This project evolved from the Energy System Insight Tool (ESIT). The ESIT was a text-based interface tool (using Tkinter) developed to help non-specialist users manipulate complex energy system models. That original model's main goal was the modeling of the energy transition in Germany, considering a desired 80% reduction in CO₂ emissions until 2050 ([BARBOSA; RIPP; STEINKE, 2021](#)).

The experience and architecture from ESIT, which was based on the LangGraph framework and the Llama 3 language model, serve as the foundation for this current thesis. However, the new scope—shifting from model simulation to real-time lab management, hardware integration (Beckhoff), and a message-based UI (Telegram)—necessitated a re-evaluation of the core Large Language Model (LLM) to better suit these new, highly-interactive and agentic demands.

3.1 Large Language Models (LLMs)

Recently, mainly due to the advent of text transformers and availability of computers with higher computational power and large-scale training data, Large Language Models have made huge improvements in the way a computer can be used to understand natural language input for the most diverse tasks. One of the main advancements that happened lately to language models was the migration from statistical to neural inference.

The process of training such a language model involves, among many other intermediate steps, the following:

- The tokenization of the training data, process that will break the text into characters, sub-words, symbols or words before the processing of the data;
- Encoding, process that transforms the data into a format understandable by the training algorithm;
- Normalization to achieve faster convergence of the model;

- PreProcessing of the data to filter out low quality inputs, duplicates and personal information;
- Pre-Training to achieve early results;
- Fine-Tuning of the model;

The reader is encouraged to check (NAVEED et al., 2023) for more details on the training process and on the types of Large Language Models.

For our original work on ESIT, the main goal was to come up with an openly accessible tool for energy system interaction, which means that the LLM and the services used should also be openly accessible. Following this precept, our model of choice ended up being Llama (DUBEY et al., 2024), an open source model developed by Meta with the goal of democratizing the access to state-of-the-art language models for anyone willing to develop tools to make use of it.

The family of Llama 3 (DUBEY et al., 2024) models is available for use in three different model sizes, 8B, 70B and 405B parameters. This value references the number of trainable parameters of the model, generally speaking, the bigger the model, the best it performs. In this case 405B would be the best performing model available, however, the 405B parameters model is only available in the version 3.1 of Llama, which is being released partially while this thesis is written. The initial development of ESIT was originally based on the Llama 3 70B, which could be partially upgrade to Llama 3.1 70B during the development process.

Beyond the parameters count, the way the models are trained is also important for their final results. The team responsible for Llama 3 was able to reach big improvements compared to the previous version, Llama 2, when it concerns training data and training methods. There was a more careful development process of the training dataset, as well as an increase in size, from 1.8T tokens to about 15T multilingual tokens. The scale of the training also got increased by about 50 times, increasing the computational power used in the process.

One last parameter that is important for our use cases is the context window, which concerns the context window of the model, or putting in other words, the number of tokens that can be taken as context at once by the model. For Llama 3 the context window is 6K tokens, while for the 3.1 version it jumped to 128K tokens, which allows more data intensive uses with the model.

According to evaluations made by the team responsible for Llama 3, the model developed by them is on par with leading language models such as GPT-4, putting it close to the state-of-the-art, while maintaining it open source. The sum up of evaluation results in comparison with other relevant models can be seen in Table 1.

For this evaluation many benchmarks were used for different goals, we won't enter in details about them in this report, for more details the reader is encouraged to consult (DUBEY et al., 2024). However, it's important to highlight one important concept zero-shot and few-shot prompting, since it's a base concept for many of the listed benchmarks. A zero-shot prompt refers to asking the model to do something it was not trained to do without any examples of how the task should be completed, in comparison to that, we also have the concept of few-shot prompting, where examples of how to solve a problem are provided in the prompt. The number in the few-shot prompting is related to the amount of examples that are provided, for example, a 5-shot prompt provides the model with 5 different examples for the solution.

Category	Benchmark	Llama 3 8B	Gemini 2 9B	Mistral 7B	Llama 3 70B	Mistral 8x22B	GPT-3.5 Turbo	Llama 3 405B	Neomotron 4 340B	GPT-4 ₍₀₁₂₅₎	GPT-4o	Claude 3.5 Sonnet
General	MMLU _(5-shot)	69.4	72.3	61.1	83.6	76.9	70.7	87.3	82.6	85.1	89.1	89.9
	MMLU _(0-shot,CoT)	73.0	72.3 [△]	60.5	86.0	79.9	69.8	88.6	78.7 [△]	85.4	88.7	88.3
	MMLU-Pro _(5-shot,CoT)	48.3	-	36.9	66.4	56.3	49.2	73.3	62.7	64.8	74.0	77.0
	IFEval	80.4	73.6	57.6	87.5	72.7	69.9	88.6	85.1	84.3	85.6	88.0
Code	HumanEval _(0-shot)	72.6	54.3	40.2	80.5	75.6	68.00	89.0	73.2	86.6	90.2	92.0
	MBPP EvalPlus _(0-shot)	72.8	71.7	48.5	86.0	78.6	82.0	88.6	72.8	83.6	87.8	90.5
Math	GSM8K _(8-shot,CoT)	84.5	76.7	53.2	95.1	88.2	81.6	96.8	92.3 [◇]	94.2	96.1	96.4 [◇]
	MATH _(0-shot,CoT)	51.9	44.3	13.0	68.0	54.1	43.1	73.8	41.1	64.5	76.6	71.1
Reasoning	ARC Challenge _(0-shot)	83.4	87.6	74.2	94.8	88.7	83.7	96.9	94.6	96.4	96.7	96.7
	GPQA _(0-shot,CoT)	32.8	-	28.8	46.7	33.3	30.8	51.1	-	41.4	53.6	59.4
Tool use	BFCL	76.1	-	60.4	84.8	-	85.9	88.5	86.5	88.3	80.5	90.2
	Nexus	38.5	30.0	24.7	56.7	48.5	37.2	58.7	-	50.3	56.1	45.7
Long context	ZeroSCROLLS/QUALITY	81.0	-	-	90.5	-	-	95.2	-	95.2	90.5	90.5
	InfiniteBench/En.MC	65.1	-	-	78.2	-	-	83.4	-	72.1	82.5	-
	NIH/Multi-needle	98.9	-	-	97.5	-	-	98.1	-	100.0	100.0	90.8
Multilingual	MGSM _(0-shot,CoT)	68.9	53.2	29.9	86.9	71.1	51.4	91.6	-	85.9	90.5	91.6

Table 1 – **Performance of finetuned Llama 3 models on key benchmark evaluations.** The table compares the performance of the 8B, 70B, and 405B versions of Llama 3 with that of competing models. We **boldface** the best-performing model in each of three model-size equivalence classes. [△]Results obtained using 5-shot prompting (no CoT). [△]Results obtained without CoT. [◇]Results obtained using zero-shot prompting. *Adapted from The Llama 3 Herd of Models, by Llama Team, AI @ Meta, 2024 (DUBEY et al., 2024)*

Another important metric that should be taken in consideration, mainly when trying to run the models locally or in a dedicated server, is the minimum requirements. For Llama 3.1, Meta asks for a modern CPU with at least 8 cores, a powerful GPU, preferably from Nvidia with CUDA and at least 16 GB of RAM for the 8B model and 32 GB for the 70B model (AI, 2024b). There were no information about the requirements for the 405B model.

3.2 Architecture Evolution: From Llama 3 to GPT-OSS

As detailed in the previous section, the ESIT project was founded on the Llama 3 70B model. This choice was driven by its open-source license and its state-of-the-art performance, which was competitive with top proprietary models at the time (Table 1).

However, the shift in scope for this thesis—from a simulation interface to a real-time laboratory assistant—introduced significant new requirements. The new agent must:

- Interface directly with industrial hardware (Beckhoff controllers) to fetch live energy data;
- Manage interaction via an external messaging API (Telegram);
- Parse unstructured maintenance logs to retrieve historical data;
- Function as an agent capable of deciding which tool to use (e.g., fetch data, check logs, or send an alert).

These requirements placed a much heavier emphasis on the model’s agentic capabilities and native tool-use. While Llama 3 is a powerful base model, its proficiency in complex, multi-step function calling (as required by our LangGraph agent) was found to be less robust than newer, specialized models.

This led to the decision to migrate the project to GPT-OSS 120B (AGARWAL et al., 2025). This model, also released under a permissive open-source (Apache 2.0) license by OpenAI, is specifically designed for high-reasoning, agentic, and production-oriented use cases. The justification for this migration is threefold:

1. **Superior Agentic Performance:** GPT-OSS was explicitly trained for native tool use, function calling, and structured output generation. This is critical for the reliability of the bot, which must correctly parse a user’s request and select the appropriate tool from its available functions.
2. **Efficient MoE Architecture:** The 120B model is a Mixture-of-Experts (MoE) model with approximately 117B total parameters but only ≈ 5.1 B active parameters per token. This architecture provides the reasoning power of a very large model while offering inference efficiency closer to a much smaller one, a crucial balance for a real-time Telegram bot.
3. **Large Context Window and Debuggability:** GPT-OSS features a 128K token context window, matching Llama 3.1. This is ample for processing extensive

maintenance logs or long conversation histories. Furthermore, it provides full chain-of-thought access, which is invaluable for debugging the complex agentic logic being orchestrated by LangGraph.

While Llama 3 provided an excellent foundation, the GPT-OSS 120B model is better aligned with the specific engineering challenges of this thesis, namely the reliable integration of hardware, APIs, and data analysis within a single agentic framework. Based on different benchmark scores, as seen on Table 2, we can also see that even using low levels of reasoning, GPT OSS surpasses or is at least on-par with Llama 3 on any metric.

Category	Benchmark	gpt-oss-120b (low)	gpt-oss-20b (low)
Accuracy (%)	AIME 2024 (no tools)	56.3	42.1
	AIME 2024 (with tools)	75.4	61.2
	AIME 2025 (no tools)	50.4	37.1
	AIME 2025 (with tools)	72.9	57.5
	GPQA Dieamont (no tools)	67.1	56.8
	GPQA Dieamont (with tools)	68.1	58.0
	HLE (no tools)	5.2	4.2
	HLE (with tools)	9.1	6.3
	MMLU	85.9	80.4
	SWE-Bench Verified	47.9	37.4
	Tau-Bench Retail	49.4	35.0
	Tau-Bench Airline	42.6	32.0
	Aider Polyglot	24.0	16.6
	MMMLU (Average)	74.1	67.0
Score (%)	HealthBench	53.0	40.4
	HealthBench Hard	22.8	9.0
	HealthBench Consensus	90.6	84.9
Elo	Codeforces (no tools)	1595	1366
	Codeforces (with tools)	1653	1251

Table 2 – Evaluation of GPT OSS 120B and 20B using low reasoning effort across multiple benchmarks *Adapted from gpt-oss-120b & gpt-oss-20b Model Card, by OpenAi Team, 2025 (AGARWAL et al., 2025)*

3.3 Text-based User Interface

Two of the main paradigms of user interface building are the Graphical User Interface (GUI) and the Text-based User Interface (TUI). They differ, as the name suggests, in the way the user interacts with the system. In the former the user is presented with a graphical environment that presents all of the possibilities, takes care of data visualization seamlessly and normally does a good job in keeping the user inside the defined limits of the program in question. By contrast, the later uses a simple text interface for the user to interact with the program, which in most of the times can be limiting regarding the amount of data that can be shown to the user, and how this data is organized.

One of the most known cases of TUI is the Command-line Interface (CLI), well known of programmers and adepts of Unix-based operational systems.

However, at which extent one is better than the other? Clearly, both have their strong and weak points that can be considered when designing a system. Normally, it's assumed that GUI is better than TUI, but according to (CHEN; ZHANG, 2007), which presented a experiment where 10 novice users and 10 expert users were given tasks to execute both on a TUI and a GUI, the difference between the two paradigms is actually not that big. The biggest variations were verified when novice users had to execute the task in the different interfaces, for them, the GUI worked a little better. However, even for novices, the ones that used the TUI before going to the GUI ended up having a good knowledge transfer from the former to the later, meaning that they had more knowledge of how the system worked than the ones who did the opposite.

Yet, this comparison was designed considering the usage of a CLI program as the TUI version, our approach makes use of a simplification layer that allows the TUI program to be much more accessible for layperson and novice users. This layer is the natural language interaction, which allows any user to simply talk to the program and ask it to execute any desired task, avoiding any direct interaction with the data, the results and any other control step normally necessary to run the models and visualize the results. (SHEN et al., 2022) and (HENDRIX et al., 1978) both talk about the benefits of adding a natural language processing layer to help users interact with high volumes of data, a great fit with energy data monitoring and its high ammount of data points.

Still supporting the idea of using LLMs to help the users to interact with data and retrieve data about the energy consumption monitoring, we cite (BOREK, 2024), (LI; APPLEBY; SUH, 2024) and (UMAR, 2024). The first showcases the usage of Retrieval Augmented Generation (RAG) together with fine-tuning of the model to create a sensitive death doula chatbot, the second demonstrates how a LLM layer can help accessing structured data from less uniform user inputs, while the third compares the user perception about search results obtained through RAG and through traditional site search functionalities. All of these functionalities got, at least in parts, integrated to LEIA.

3.4 Consumption Data Gathering

In order to gather the power consumption and associated data, we combined hardware from Beckhoff and a series of protocols. The Beckhoff hardware setup is as follows:

- **CX8190:** embedded PC that collects measurements and sends them over the Internet;
- **PS1011-2410-0000:** power supply for the setup;

- **EL3433:** power measurement terminal;
- **SCT1111-0040:** ring-type current transformers (three units).

The current transformers are hooked up to each circuit of a laboratory distribution panel, and to the EL3433 terminal.

The data, once collected by the CX8190 unit, is packaged into a JSON string and sent to an external MQTT broker. A subscriber, connected to that same broker, collects the messages and sends them to the HTTP API, which saves everything to an SQLite database. LEIA accesses the data by making requests to that API.

MQTT was chosen due to its lightweight nature and the fact that the TwinCat 3 software used to program the CX8190 unit already has an MQTT library. However, HTTP was also used so as to allow more complex queries on the data to be communicated by LEIA — neither protocol could have been used alone.

The implementation and underlying challenges faced during the implementation of the physical system are explained in details in Section 4.1, Section 4.2 and Section 4.3, under Chapter 4.

3.5 LangChain

LangChain is today one of the most prominent open-source libraries for developing LLM-powered applications, being then the chosen one to be the base of ESIT. It allows among other things, the building of LLM agents through building blocks which connect like a chain, as suggested by the library’s name. Some examples of possible agents that can be built using LangChain’s building blocks are shown in Figure 1.

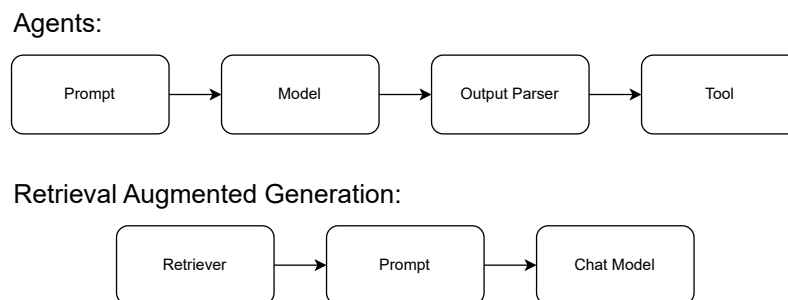


Figure 1 – Example chains that can be built using LangChain.

The LangChain library also offers another useful module which is called LangGraph, dedicated to allowing the developers to build applications based in state graphs, making use of many different agents for different purposes inside of the pipeline. As we will better

see in Chapter 4, this is the module that we used to build LEIA, since it allows more control of the data and control flow of the applications.

Some prior works also make use of LangChain for similar purposed tasks, such as building a context-aware and personalized chatbot for customer service (PANDYA; HOLIA, 2023), a chatbot tailored for helping the mental health care of users (SINGH et al., 2024), and even providing initial analysis of literature review for specific research fields (JEONG et al., 2024). For a broader analysis of LangChain’s capacities on developing LLM-based applications, the reader is encouraged to read (TOPSAKAL; AKINCI, 2023), which analyses each of the core features of LangChain and their integration capacity, as well as providing practical examples.

3.6 GroqCloud

Since the requirements for running GPT OSS 120B and 20B locally are considerably high, as seen in Section 3.1, we decided to, instead of relying on a local instance of the model, actually make use of one of the available cloud services hosting language models. The one selected for this project was GroqCloud (GROQ, 2024b), an online service that provides access to a gamma of different state-of-the-art models such as Google’s Gemma, Mistral’s Mixtral, Meta’s Llama 3 and 3.1, and OpenAi’s GPT OSS 120B and 20B (GROQ, 2024a).

The Groq’s servers for LLM inference use a special type of processing unit, a Language Processing Unit (LPU) developed by Groq itself as a processor dedicated to language models, able run LLMs faster and with better energy efficiency than conventional Graphics Processing Units (GPU) (GROQ, 2024c). The processing speed of LLM requests really comes in hand when we start to pile up many different agents to build the application pipeline.

Another reason for Groq to be chosen was the availability of a rather generous free tier subscription plan. Even with the free plan we were able to implement a tool that executes many requests per execution, consuming thousands of tokens per use. The current limits for the models used in LEIA on the free plan and on their pay-as-you-go developer’s plan are available in Table 3.

3.7 Tavily

One of the ways to avoid hallucinations using LLMs resides in allowing the agents to search for information online instead of relying completely on the training data. This allows not only more precise answers but also real time information, such as information about the weather, traffic and current events. To include this capability in our tool we

integrated Tavily API ([TAVILY, 2024](#)) as a web search provider. Tavily allows for API requests to find information online that can be then used as context for the AI. Beyond that, Tavily is cited by LangChain as one of their essential partners, offering even native support for it. The last reason that made us choose Tavily as our web search provider was the free tier plan, which includes a total of 1000 requests per month with no costs, more than enough for our test cases.

3.8 Retrieval Augmented Generation (RAG)

One of the most important components of any LLM-based tool that works with a defined set of source files or databases is a Retrieval Augmented Generation (RAG) agent. As the name suggests, this agent is capable of generating outputs based on file retrieval. In many cases where there is not enough data for a fine tuning of the model, or when the base data changes frequently, it's possible to implement an RAG agent to retrieve information from the available sources. With the retrieved context from the files the LLM is then able to use that to answer the user about any case specific question.

ESIT made use of RAG to provide the model with information from the model's source paper ([BARBOSA; RIPP; STEINKE, 2021](#)) and from the solver's documentation ([BARBOSA, 2023](#)). That allows the application to answer the user about specific information about the goals of the model, the methods used in the modeling and the specific modeling details of the integrated solver.

LEIA doesn't actually use RAG for anything specific, however, we decided to keep the RAG module so it would be capable of using it without big efforts if necessary in another use case in the future.

To create and populate RAG module, we have some steps to prepare the data for the language model to consume. Having the data loaded, we need to complete the following steps to make it available for the model:

Model	Tier	RPM	RPD	TPM	TPD
Llama 3.1 8B Instant	Free	30	14.400	6.000	500.000
	Developers	1.000	500.000	250.000	-
Llama 3.3 70B Versatile	Free	30	1.000	12.000	100.000
	Developers	1.000	500.000	300.000	-
GPT OSS 20B	Free	30	1.000	8.000	200.000
	Developers	1.000	500.000	250.000	-
GPT OSS 120B	Free	30	1.000	8.000	200.000
	Developers	1.000	500.000	250.000	-

Table 3 – Current limitations of model usage in Groq's free plan

- **Parsing:** before anything, it's important to parse the documents to a format more understandable to the model. For example by converting tables to a format the AI can actually read, or organizing the text structure to be more linear;
- **Indexing:** with the parsed document available it's necessary now to convert the data to a structure format that allows for querying easily. Normally that is solved using vector embeddings that convert the text into a numerical representation of the data, allowing for example searches by vector similarity;
- **Storing:** the vector embeddings should be stored to allow them to be re-utilized.

Only after storing the embedded data we can finally execute queries that will be able to consult this database and provide context to the AI.

To manage all of these steps we use a range of libraries and frameworks, as well as external services:

- **LlamaIndex (LIU, 2022a):** Framework responsible for most of the necessary tools for building context-augmented generative AI, it provides tools such as vectorizers, embedders, storage access, querying, among others;
- **LlamaParse (LIU, 2022b):** Developed by the team responsible for LlamaIndex, is a cloud based parsing tool that allows up to 1000 pages per day in the free plan, more than enough for our use;
- **Qdrant (QDRANT, 2024):** Cloud vector database responsible for storing and accessing the embedded data used by our model, it also offers a free plan with 1 GB of storage as well as basic processing power availability.

3.9 Prompting strategies

While developing an LLM based application, one of the most important steps is to write effective prompts that will allow the language model to correctly execute the tasks it was designed to. This step helps to steer the tool to the right execution paths even when the user's prompts are not ideal, non-AI-experts tend to overgeneralize their prompts, mostly expecting something closer to a human-to-human interaction, thus reducing the effectiveness of said prompts (ZAMFIRESCU-PEREIRA et al., 2023). Another factor that may lead to further reductions in the prompt effectiveness are related to the way this prompts are written, a study (OPPENLAENDER; LINDER; SILVENNOINEN, 2024) has shown that users are normally able to evaluate and design prompts that normally end up failing to use some necessary vocabulary and phrasing strategies that lead to better prediction results. In that sense, to allow the users to struggle the less as possible with

prompt designing we need to prepare the prompts with instructions and examples for them to better receive the user input. Many prompt strategies can actually help us in achieving this goal.

Few-shot and 0-shot prompting are two of the base types of prompt that define the way a model will tackle on the problem. In the first, the model will be presented some examples, called "shots", of how the problem should be solved for it to base its prediction, this strategy allows the model to work even on cases that were not covered by its training dataset. On the other hand, in the second there are no examples and the problem is directly presented to the model, who becomes fully responsible of understanding and solving it based solely on its pre-existing knowledge. A further description of both methods can be seen in (LI, 2023).

The strategy used for LEIA is the 0-shot discrete prompting, in which a instruction is given to the model explaining how to perform a specific task, but no further information is provided, hence 0-shot. This helps to prevent hallucinations and provides more predictability to the model outputs, allowing for easier integration between the different nodes of the bot.

However, a few core nodes of the tool also use few-shot learning techniques to provide a better control of the possibilities that can be presented to the model. This allows us to avoid taking on extensive and data-intensive fine-tuning processes while maintaining good and consistent results. A study conducted by OpenAI, the company responsible for GPT, has shown that language models in which few-shot learning was applied often were on par with fine tuned models designed for the same task (BROWN et al., 2020).

Beyond these two strategies, there are also some paradigms that can be used to improve results on LLM predictions. These are mostly used for reasoning tasks and for this reason ended up not being incorporated in our project, however, their importance for other applications is notable. Some of the most prominent paradigms used today are **Chain-of-Thought (CoT)** (WEI et al., 2022), **Reasoning and Acting (ReAct)** (YAO et al., 2023), **Explain-then-Predict (E-P)** and **Predict-then-Explain (P-E)** (YE; DURRETT, 2022).

Chain-of-Thought (CoT) bases its strategy in encouraging the model to break down its reasoning for the problem into smaller steps, instead of requesting for a direct answer. This paradigm becomes really useful for math problems, logic, puzzles and more complex questions. Doing so reduces a complex problem into some smaller parts, leading to better results.

Reasoning and Acting (ReAct) adds a new layer to CoTs reasoning capability by allowing the model to also take actions. The model is prompted to, in some steps, reason about a part of the problem and then take action to gather the necessary information,

running this combination of reasoning and acting for as long as necessary to reach the final answer. This paradigm is really useful for situations where gathering external information is necessary to answer the user.

Explain-then-Predict (E-P) and **Predict-then-Explain (P-E)** both are based on adding an explanation to the answer given by the model, mostly hoping for the LLM to predict a better result by having to explain how it solved the situation, where they only differ in the order of generation. However, this simple inversion is already capable of altering the prediction results, since the explanation generated can really influence the desired output. If it comes first, the model can end up using it to modify the output, while that can be avoided by generating the explanation after prediction the answer.

4 Implementation

The implementation of LEIA encompasses two different sides, the physical solution, represented by the development of the data gathering system of consumption data from the laboratory, using the available Beckhoff devices, and the development of the agent itself, which is mainly based on the Langchain framework, using GPT OSS 120B and 20B as the LLM-engine to run the user queries.

To integrate the two sides of the project and make the actual laboratory data available to the final user, we have developed a state-controlled pipeline to handle a range of possible input types asked. Some of the possibilities that the tool can gracefully handle are:

- Handling user inputs that require multiple step solutions;
- Solving simple and complex mathematical problems;
- Finding relevant information online;
- Consulting and manipulating the maintenance history;
- Consulting the historic data from the database;
- Generating relevant plots based on the user's request;
- Understanding of audio and images;
- Multilingual input and output.

The implementation of the original project in which LEIA is based, ESIT, led us to a more refined solution. So, to explain LEIA's final state, we need to first introduce the ideas with which ESIT was developed. For the tool to be able to solve a range of problems in a multi-step configuration we had to define task specific nodes that generate the necessary context from previous nodes and for the consumption by other consecutive nodes. In this chapter we'll enter in details on how the tool was constructed, starting from the initial ideas and tests, going all the way to the final structure of ESIT, and finally explaining the evolution from ESIT to LEIA.

The final version of ESIT can be accessed by visiting the project repository at [<https://github.com/gu-trivelatto/ESIT>](https://github.com/gu-trivelatto/ESIT), while the final version of LEIA is available at [<https://github.com/gu-trivelatto/LEIA>](https://github.com/gu-trivelatto/LEIA).

4.1 Devices and Protocol

The fundamental hardware infrastructure for the monitoring system relies on the physical capture of electrical quantities and their subsequent digitization and transmission. The starting point for data collection is the ring-type current transformer (CT) sensors. These devices operate on the principle of electromagnetic induction, allowing for non-intrusive measurement of the current flowing through the laboratory circuit conductors.

The analog signals generated by the current transformers are received by a dedicated measurement terminal. This terminal is responsible for analog-to-digital (A/D) conversion and signal pre-processing, transforming the induced currents into interpretable digital data. The terminal communicates directly with the central processing unit, a Beckhoff CX8190 controller.

The CX8190 is an ARM-based Industrial PC (IPC) running the Windows operating system. This device acts as the "brain" of the local automation, orchestrating terminal readings and control logic. To develop and execute the control logic within this environment, the TwinCAT 3 software was utilized. The TwinCAT environment allows the industrial PC to function as a real-time Programmable Logic Controller (PLC). The programming language used for developing the code on the PLC was Structured Text (ST). ST is a high-level language, similar to Pascal, and is part of the IEC 61131-3 international standard for industrial automation. This was necessary due to limitations and requirements of the Beckhoff devices.

For external communication and interoperability between the Operational Technology layer and the high-level Information Technology software system, the MQTT (Message Queuing Telemetry Transport) protocol was adopted. MQTT is a lightweight, publish/subscribe messaging protocol, ideal for IoT environments where bandwidth and latency are concerns. The Mosquitto software was employed as the MQTT broker, acting as the intermediary that manages message queues between the PLC (publisher) and the subsequent software systems (subscribers).

4.2 Data Gathering

The Data Gathering process is divided into two main stages: the internal logic of the PLC and the data subscription and handling service.

On the PLC side (CX8190), the implementation in Structured Text overcomes the language's expressiveness limitations through a cyclic task architecture. The code defines a custom data structure designed to store all relevant electrical metrics, including voltages, currents, power (active, reactive, and apparent), and power factor for each monitored sensor. The operation cycle functions as follows:

- **Acquisition and Buffering:** A high-priority periodic task reads the raw data from the measurement terminal's memory structure. This data populates the defined struct, which is then inserted into a circular buffer. The use of a buffer is crucial to ensure that no samples are lost in the event of network processing latency.
- **Serialization and Publication:** A second task consumes this buffer of complete structs. Using a library specific to the TwinCAT environment, the system converts the structured data into a JSON string. Subsequently, this JSON string is published to a pre-defined topic on the MQTT broker.

Once the data leaves the industrial PLC environment via MQTT, it is captured by a software component developed in Python, designated as the Subscriber. This component utilizes the `gmqtt` library to maintain a persistent connection with the broker. Upon receiving the raw JSON messages from the PLC, the Subscriber performs domain-specific processing: it unwraps the aggregated data and restructures it into individual objects for each sensor.

For each sensor reading, the system generates and sends a standardized JSON object to the API containing: channel and sensor identification, power metrics, power factor, current, voltage, and the timestamp. The actual format of the JSON object can be seen in Listing 4.1.

Listing 4.1 – JSON object of the readings

```
{
    channel: string ,
    sensor: string ,
    apparent_power: number ,
    active_power: number ,
    reactive_power: number ,
    power_factor: number ,
    current: number ,
    voltage: number ,
    timestamp: string
}
```

The final data flow representing the data gathering implementation can be observed in Figure 2, including the visualization of each of the previously described devices and systems.

It is important to note that the semantic integrity of the data—identifying which sensor corresponds to which physical laboratory circuit—required a rigorous manual mapping process. This involved physical inspection of the distribution board, controlled

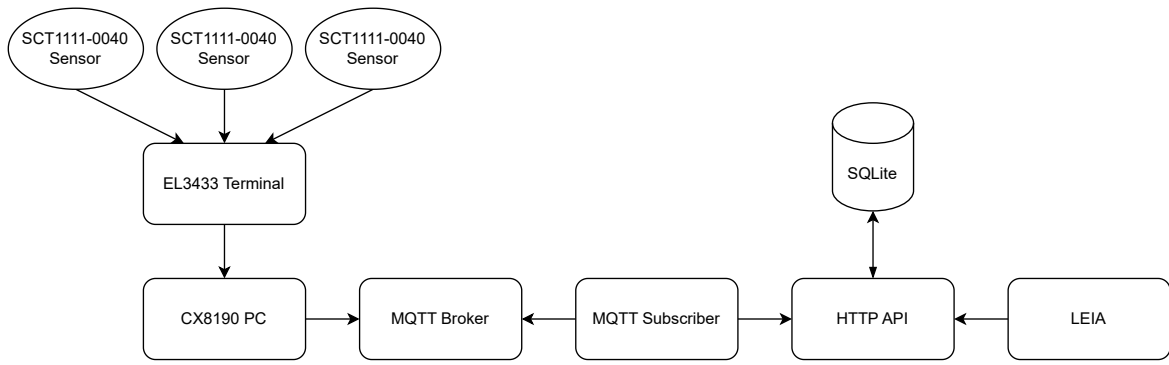


Figure 2 – Representation of the measurement data flow.

opening of circuit breakers to observe load interruption in real-time data, and verification of circuit labels. This mapping was hardcoded into the system logic to ensure that the digital data accurately reflects the physical reality of the laboratory’s phases and circuits. Therefore, the addition of new sensors requires updates to both the PLC code and the Subscriber’s restructuring logic.

4.3 Data Availability

The final layer of the implementation focuses on data availability and persistence, ensuring that collected information is accessible to the LEIA assistant and other analytical tools. This layer was built upon a microservices architecture hosted on the same physical machine running the MQTT broker and the Subscriber, facilitating local communication.

The core of this layer is a RESTful API developed using the FastAPI framework. The choice of FastAPI is justified by its high performance, asynchronous nature, ease of documentation, and native integration with data validation. For persistence, SQLite was chosen as the database. Although it is a lightweight file-based database, SQLite offers the necessary robustness for the project’s data volume without the complexity of managing dedicated database servers (such as PostgreSQL or MySQL).

Data integrity, both at input (data coming from the Subscriber) and output (data served to LEIA), is guaranteed by the Pydantic library. Pydantic enforces strict data typing, ensuring that the system rejects invalid or corrupt formats before they are persisted or processed. HTTP communication is managed by the Uvicorn ASGI server, while an Apache2 server acts as a reverse proxy, providing an additional layer of security and traffic management.

The API exposes a series of specific endpoints designed to provide granular insights into energy consumption. The list of implemented endpoints includes:

- `/analytics/channel/consumption`: Returns accumulated consumption data.

- `/analytics/channel/demand_peaks`: Identifies demand peaks within time intervals.
- `/analytics/channel/electrical_health`: Provides metrics related to power quality.
- `/analytics/channel/hourly_profile`: Generates the average hourly consumption profile.
- `/analytics/channel/current_by_sensor`: Details specific current by individual sensor.
- `/analytics/channel/voltage_anomalies`: Reports voltage fluctuations outside nominal parameters.

This architecture decouples the collection logic (PLC/MQTT) from the consumption logic (Chatbot/Analytics). The API serves as the single source of truth, abstracting the underlying hardware complexity and allowing the LEIA agent to perform complex queries through simple HTTP calls using the `httpx` library.

4.4 Agent development

4.4.1 Initial idea: multi-tool with LangChain

The initial idea tested was to try to make use of tooling capacity of the Langchain and Llama to allow a single agent to decide the necessary tools, execute them and decide if the result was already sufficient or not. This was done by setting a generic prompt explaining to the agent how it should behave.

The basic structure of this prompt consisted of a brief explanation of what was the goal of the agent, what were the available tools (described in a JSON format with input and output parameters, being them a RAG retriever, a web search, a calculator and a date retriever), and the way it should output the answers until the final response was generated.

The output scheme of the agent followed the chain-of-thought prompting ([WEI et al., 2023](#)) to try and enhance the reasoning of the LLM by outputting a series of intermediate steps while the answer was not ready.

The full prompt can be found in the [Appendix A](#). After running some initial tests it was already clear that, although the LLM actually followed a pretty straight forward reasoning for the output generation, the available models still didn't have the necessary support for tooling. The model consistently hallucinated the answer instead of using the available tools, rendering the tool basically unusable, since the output results were most of the times false.

However, it's important to notice that at the time in which these tests were made Llama still didn't have any model actually compatible with tooling. While this report is being written a GPT-OSS with tooling capacities is already available and could find it's way into solving the problems found during this early tests of ESIT.

This approach can be called fully autonomous agent, as explained in ([WOODSIDE; TONER, 2024](#)), since it had the freedom of, considering the provided possibilities, decide the way to break down the proposed problem in smaller parts, which tools to use and what information to provide them, as well as deciding when it was ready to answer. In a scenario like this we basically have no control of the agent whatsoever beyond that of explaining it its goals and providing the available tools for it to use. In contrast with that, we delved into the usage of LangGraph to find a sweet spot between system autonomy and data flow control. The next section explains in details the development process for the current architecture of ESIT.

4.4.2 Final idea for ESIT: node tree with LangGraph

The decision to adopt LangGraph stemmed from the need for a more controllable and modular way of running the system's tools. Instead of relying on a single autonomous agent to decide all actions, the new approach decomposed the workflow into smaller, specialized agents connected sequentially within a graph structure. This allowed for clearer execution control, conditional logic, and feedback loops when necessary.

In early prototypes, the focus was on understanding LangGraph's operation through a small set of control nodes and tools. In Figure 3 we can see the original implementation. It is described by a flow starting in the user input, followed by the **Date getter** node to enrich the agent context about the current data. With the enriched input, we start the response iteration. By analyzing from the context gathered in each iteration, the **Context analyzer** decides if the agent is ready to answer or not, after each action taken. If the router decides that the answer requires more context, it flows then to a **Tool router** that decides which tool needs to be used to gather the missing information. We had a set of 4 tools available, a **RAG** module, a **Web Search** module, a simple **Calculator** module and a **Model Modifier** module used in by the energy system module use case. Finally, the **Output Generator** would use all the context generated during the iterations to generate the correct response to the user.

Although this first version was rudimentary, it provided valuable insight into how to structure the data flow and decision-making logic across nodes and edges.

Building on these experiments, the project evolved into a more domain-oriented architecture focused on energy system interactions (Figure 4). The graph was divided into three branches:

Initial graph:

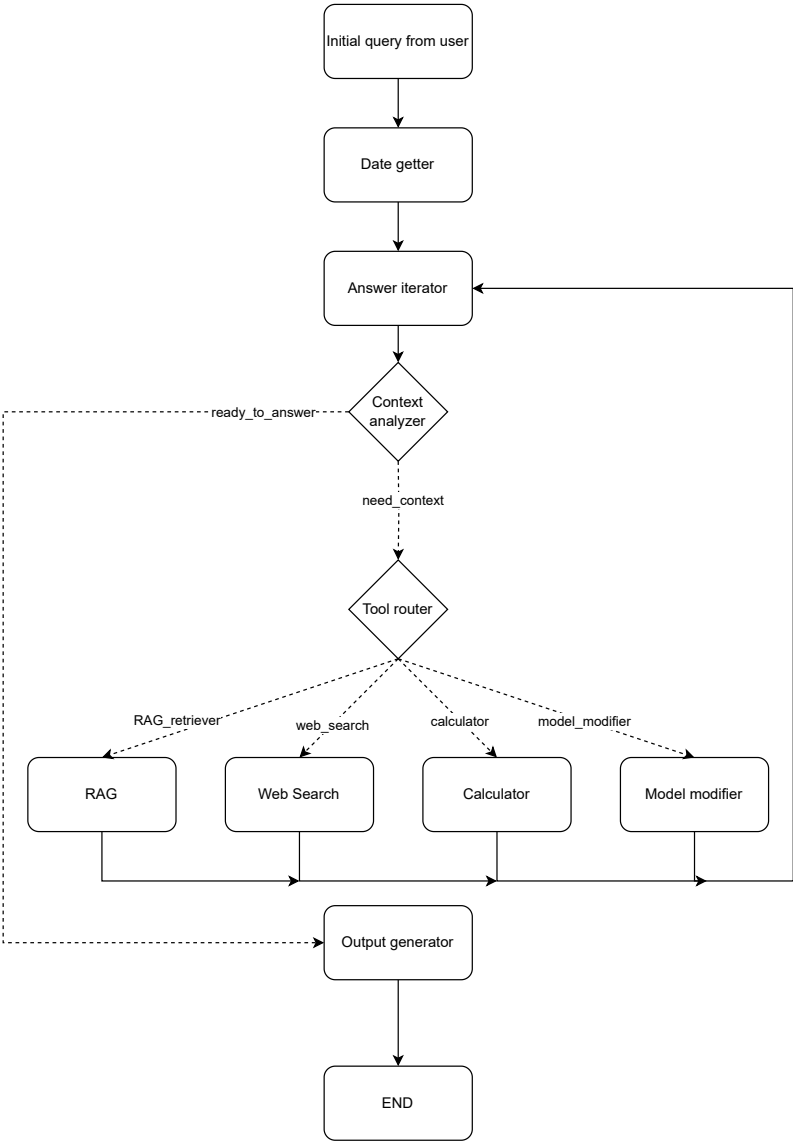


Figure 3 – Initial version of the graph.

Complete initial graph:

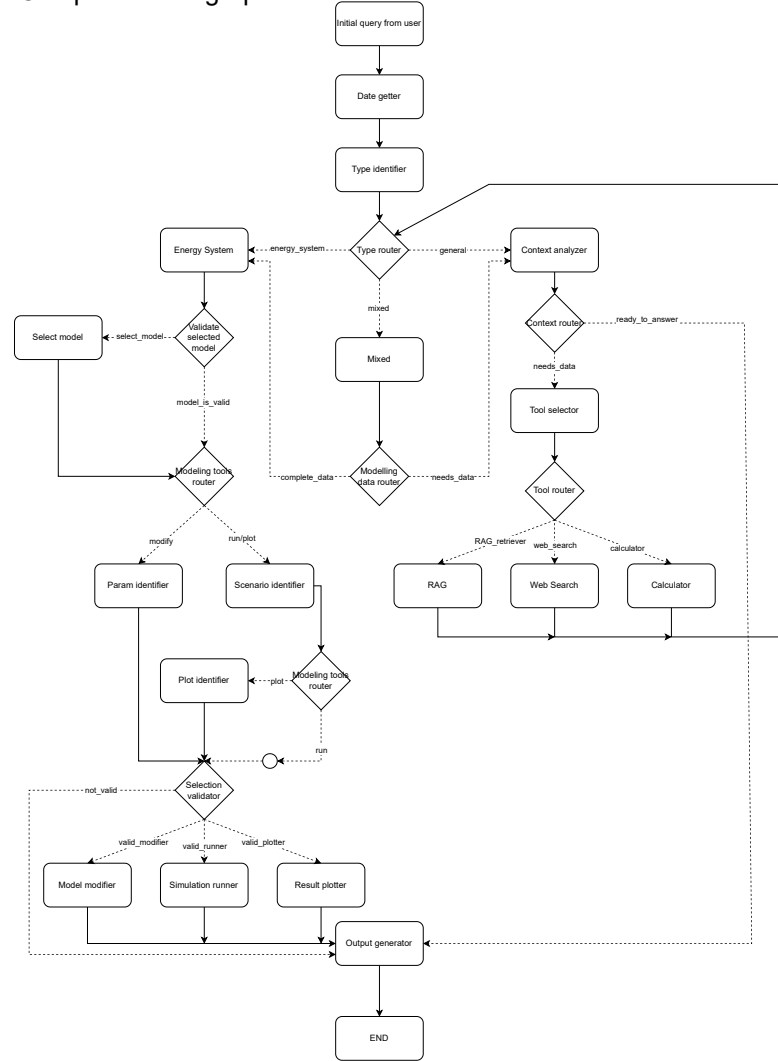


Figure 4 – Initial version of the final graph.

1. **Energy system branch:** for modeling, simulation, and parameter management.
2. **General branch:** for information retrieval and simpler tasks.
3. **Mixed branch:** combining both, when external information was needed before executing modeling actions.

However, the large number of possible paths and dependencies made this version overly complex and reliant on user input. This led to a major redesign that produced a simplified and more autonomous final version (Figure 5).

The final graph integrates an input consolidator to gather context from conversation history and an action selector to determine necessary energy-system operations. These

improvements streamlined execution, reduced branching, and enabled more consistent autonomous behavior across agents.

The different branches have some specific nodes to treat each of the use cases. The **Energy System** branch operated in the same iterative context gathering flow, but focusing on modeling. We broke the modeling tool into different tools to reduce the general scope. **Consult Model** is responsible for getting the actual model parameters, **RAG** is responsible for consulting theoretic data about the model, **Modify Model** is responsible for changing the model parameters, **Run Model** runs the modified model, **Compare Model** creates a textual comparison of results from two different model runs and finally, **Plot Model** plots the results for the user to see.

The **General** branch kept the remaining simple tools, **Web Search** and **Calculator**. The **Mixed** branch is used when it is necessary to get information from the **General** branch before acting on the model itself, having specific routers and context analyzers to decide when to re-route to the final iterations.

Complete final graph:

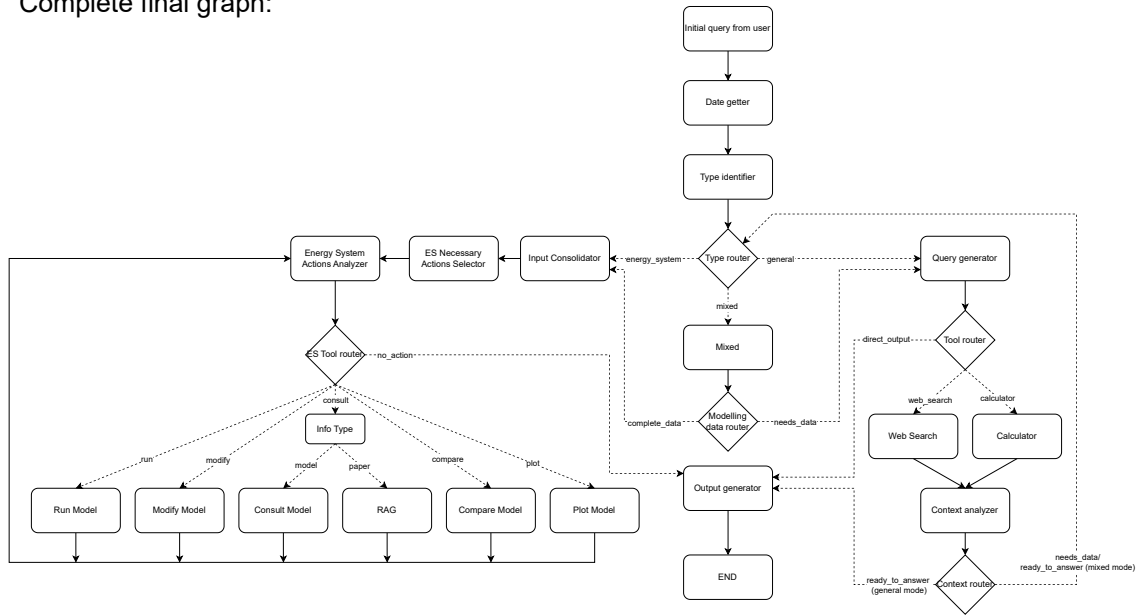


Figure 5 – Final version of the graph.

4.4.3 LEIA improvements and final form

Following the development of ESIT, we could notice that the most complex part of the pipeline was its management. Deciding what, when, and how the bot should do every time, and deciding that based on the outputs of many different LLM executions was undoubtedly hard. However, the release of GPT OSS (AGARWAL et al., 2025) in August 2025 unlocked many resources that were unavailable to the open weight community prior to it. LEIA has greatly improved ESIT from its original implementation, mostly by

migrating from Llama to GPT OSS. This seemingly simple migration has enabled us to develop a more concise and simple workflow for the bot execution, which doesn't rely on support agents and complex coordination algorithms to reach results that are even better and more predictable.

We were able to simplify the bot graph by using a single LLM node that is tied to all necessary tools and is capable of correctly and accurately calling them whenever the user requests something related. The graph is now focused on working around the LLM agent itself, so now, we simply treat the input to make it LLM-friendly, run the inference, and finally, format the output to a desired shape. By checking the diagram in 6 we can better understand what each of the nodes are responsible for. We still have other LLM models running together, but apart from the main bot, the other models are focused on message treating, and not the flow itself.

The complete flow begins when Telegram sends a webhook notifying the system that the bot has received a new message. This event triggers the response-processing pipeline. Before the LLM workflow starts, the input must be preprocessed. This preprocessing layer includes two auxiliary models used to handle audio and image messages: Whisper Large V3 Turbo and Llama 4 Scout 17B, respectively. Whisper is a speech-to-text (STT) model specialized in converting audio into plain text, while Llama 4 Scout is an omni-channel LLM capable of interpreting different input modalities. Its main role in this context is to provide an accurate description of an image so that the downstream response-generation workflow can correctly interpret its content.

The first node, `input_digest`, is responsible for preparing the input by appending it to the conversation history and limiting what is forwarded to the LLM. The `main_bot` node represents the primary LLM agent, which processes the input, generates the response, and may use any of its available tools—such as a web search, or data access and maintenance history access. Finally, the `formatter` agent takes the generated response and splits it into multiple messages, formats URLs, and applies general text styling. It outputs a JSON-parsed structure, which is ultimately sent back to Telegram for delivery to the user. The tool usage and definitions are explained in Section 4.4.4.

4.4.4 Tools

As previously described, LEIA provides three main tools used throughout the workflow execution: web search, data access, and maintenance history access. All three are designed to gather information to be returned to the user, each sourcing data from a different location.

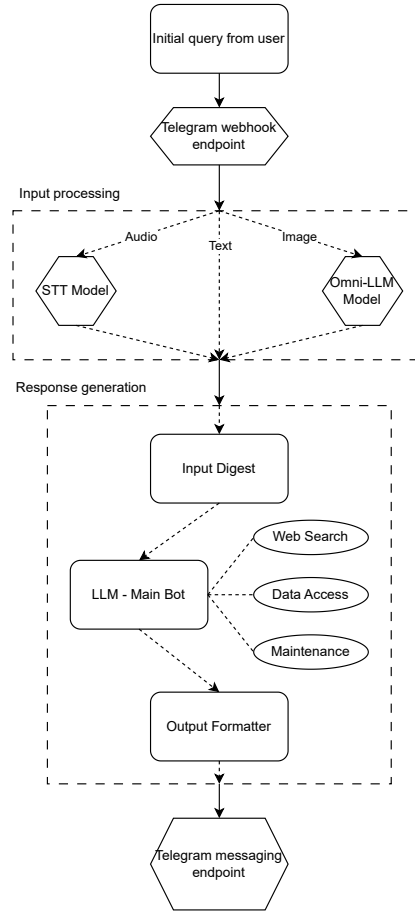


Figure 6 – LEIA’s graph simplified by using GPT OSS.

4.4.4.1 Web Search

As explained in Chapter 3, the web search tool is powered by the Tavily API, an online service that enables web searches through textual queries, functioning similarly to any standard search engine.

Although the tool supports multiple configuration parameters to narrow search results, our use case involves a wide variety of possible query types. Therefore, we only limited the maximum number of results returned. These results are then passed to the agent, which extracts the most relevant information from the retrieved websites.

The agent is also explicitly instructed to always include the source from which the information was obtained.

4.4.4.2 Data Access

Beyond retrieving data from the database and summarizing it to help the LLM understand the current context, this tool is also responsible for generating the visualizations required by user queries.

This tool works with two inputs: the requested insight (from a predefined list) and the time period (specified in natural language). The available queries are: total electrical consumption, consumption peaks, electrical health (power factor), hourly consumption profile, balance of current between phases, and voltage anomalies.

Corresponding database queries were then written into endpoints of the measurement data API (discussed in Section 3.4), with only minor computation being done on the tool-calling side after the data is requested. The exception to that is plot generation. Generated plots are stored locally, and later accessed by the application to send the image to the user after the agent finishes processing the response.

4.4.4.3 Maintenance History Access

This tool is responsible for interacting with an online maintenance history spreadsheet hosted on Google Sheets. To enable access to this sheet, we created a project within Google Cloud and configured a service account with the necessary permissions.

Using Google authentication, the tool is able to read and manage the maintenance history: retrieving the latest completed tasks, listing scheduled ones, updating existing entries, and creating new scheduled jobs.

4.4.5 Prompt engineering

Since ESIT was based on Llama, a model without reliable tooling capacity, almost each of its graph nodes include at least one LLM chain consisting of a prompt, the model and an output parser (excepting some nodes such as the date getter and the model runner, which simply execute the tasks). Llama utilizes a specific format for prompt building that must be followed for the LLM to understand correctly how to execute a given task. This format comprises the usage of some specific tags to indicate some key information about the prompt to the model, some of the specific keys are (adapted from (AI, 2024a)):

- `<|begin_of_text|>`: specifies the start of the prompt
- `<|start_header_id|>` and `<|end_header_id|>`: indicates the role for a particular message. The available roles are `[system, user, assistant, ipython]`
- `<|eot_id|>`: used at the end of interactions between the model and the user, or at the end of interactions with any tools to indicate that it finished the interaction

In Listing 4.2 we have a brief example of one of the prompts developed for the application. It is used for the calculator chain, where the model should decide, from the user input, what is the mathematical equation it should use to solve the provided problem. As can be seen, we firstly provide the **system** prompt, which will explain to the agent how it should behave, in this step we should set the rules, explain how the tool it's using works and limit the outputs it should generate to fit the format that we need. In this case (as in many other prompts used in this project) we request a JSON output with the desired output, since this is much more controllable than a simple textual input. After providing all the necessary instructions, we then start the **user** section, with the variables such as the user input and in this case the context information that may be necessary to solve the mathematical equation, and finally, we start the **assistant** section to signal that the model should start generating.

Listing 4.2 – Prompt for the calculator agent

```
"" <|begin_of_text|><|start_header_id|>system<|end_header_id|>
  You are an specialist at identifying the correct operation
    and operands to build
  a JSON object with the correct equation to be solved , you
    will get the information of
  what equation to generate from the given QUERY. \n

  You must output a JSON object with a single key 'equation',
    that must be in a format
  solvable with Python, since we'll solve this using the eval
    () function. You don't need
  to import anything or prepare the environment in any way,
    the environment is ready ,
  you just need to build the equation in a format recognizable
    by python, using python
  mathematical functions if necessary. \n

  NEVER USE IMPORT STATEMENTS, YOU MUST ASSUME THAT ALL
    NECESSARY LIBRARIES ARE ALREADY
  IMPORTED, JUST USE THEM. \n

  If you need information that you don't know to build the
    equation you may find that
  in the CONTEXT, so check it for more details on the data. \n

  If you need intermediate results , you can calculate only
```

```

    them and leave the final
    result for later , you're part of a iterative tool that may
    be called as many times
    as needed to find the final result . \n

```

```

Always use double quotes in the JSON object . \n

```

```

<|eot_id|><|start_header_id|>user<|end_header_id|>
QUERY: {query} \n
CONTEXT: {context}
<|eot_id|><|start_header_id|>assistant<|end_header_id|>""

```

Any other prompt in the system is also developed using the same main features, a full explanation of how the agent should behave, define any necessary constraints, define the output type, include the necessary variables and then it's ready to be connected to the LLM chain.

LEIA allowed us to break all of this complexity into a more concise and elegant solution, first of all by removing all different agents and prompts, we now have a single agent with the whole context of its functionality and some tools with descriptions that help the agent to decide what to use based on the context. Second, GPT OSS also allows for a more simple and direct history and messaging format, since it uses OpenAi base format for message organization. It's is based in a simple array of three message types, **system**, **user** and **assistant**. By providing the LLM model with an array containing initially the prompt as a **system** message, and then a sequence of **user** and **assistant** messages for history, we can easily define the conversation for which the model will run the inference. In Listing 4.3 we have an example of this array.

Listing 4.3 – Memory example for GPT OSS

```

messages = [
    {
        role="system",
        content="You are a helpful assistant "
    },
    {
        role="user",
        content="Hi, how are you?"
    },
    {
        role="assistant",
        content="I'm fine , and you?"
    }
]

```

```
    }
]
```

4.5 State

As commented before, LangGraph is based on state graphs, up until now we only entered in details about the graph, however, the state is possibly the most vital component of the graph integration. In ESIT, since we had to go over different decision making agents and control the outputs of each of them, we had a complex state dictionary that contained message history, intermediate step results, routing decisions, execution types and tool outputs. With LEIA's simplification of the graph, we were able to reduce the state space and also break it in 3 different state schema for better understandability:

- **InputState**: used by the `input_digest` to input data from the graph to the LLM
 - `chat_input`: processed input sent by the user through Telegram
 - `chat_id`: ID that identifies the user chat in Telegram
 - `phone_number`: user's phone number used in Telegram
- **OutputState**: used by the `formatter` to define it's output format
 - `formatted_output`: output formatted to be sent to the user
- **MainState**: is the actual state of the graph, aggregating input, output and history
 - `chat_input`
 - `chat_id`
 - `phone_number`
 - `formatted_output`
 - `messages_history`: messaging history between the user and the agent

4.5.1 Chat history

Whenever we use any service that provides access to a LLM such as ESIT, LEIA or ChatGPT, we expect to be able to carry the conversation as we would do with any other person. That means that we assume that any past message will be remembered by it for us to reference again. One example of the expected behavior of interactions with LLMs can be seen in Listing 4.4, where the user asks for the weather, and subsequently asks, without any further context, "What about tomorrow?". LLMs are unable to remember interactions by themselves, since they are static models that runs in instances based on

the input prompt that they receive, because of this, it is important to develop a way of providing the model with the history when designing any LLM-based application.

That allows the user to interact more naturally with the application, which by itself reduce the time to get used to it. However, due to limitations we cannot simply store everything that was said during a session and input that to the agents on each new iteration, token limits and the size of the context window may cause a history that is too big to reduce the comprehension of the agent or even cause a failure on the system. Even worse, the history that could help with providing context for past interactions of the same subject may end up causing trouble when the user switch to a new task (GUPTA et al., 2024). In that regard, some studies present solutions to the necessity of accessing the model memory selectively, effectively avoiding mixing past messages related to different tasks with the current ones, some of the solutions include using a vector store, in a similar way of what is done on RAG search (HATALIS et al., 2023), using temporal event graphs to map the past interactions (MAHARANA et al., 2024) and even applying fine tuning to personalize the used LLM with memory of past interactions (ZHANG et al., 2023).

Listing 4.4 – An example of the expected interaction with a history-aware LLM

USER:

How is the weather in my area today?

ASSISTANT:

Today's temperature will vary between 13C and 22C.

USER:

What about tomorrow?

ASSISTANT:

Tomorrow high will be 24C while the low will be 17C.

LEIA's new base system supports two different memory designs. The first and simplest is in-memory storage, which is ideal for testing and evaluating the system, as it requires virtually no setup and is fully ephemeral. The second, and most suitable for production, is PostgreSQL-based memory. LangChain provides several nearly ready-to-use checkpointers, with PostgreSQL being one of them. This checkpointer is responsible for creating the memory schema in the provided database and for storing all interaction history. With PostgreSQL, we are able to persist chat memory indefinitely, ensuring that a user can return even months later and still be recognized by the chat bot, being this the memory strategy currently being used for LEIA.

Our tool uses a simple memory strategy that provides the 5 most recent messages to the user to avoid malefic effects of inputting too much context, which provides already

reasonable results, however, this represents one of the main improvement points that will be further discussed in Chapter 7.

4.5.2 Messenger Integration

Telegram offers a wide range of open-source applications ([Telegram Messenger LLP, 2025](#)), which represents a significant advantage for the development of a system such as LEIA. This openness allows us to build upon publicly available and community-maintained APIs, ensuring transparency, flexibility, and long-term sustainability. Furthermore, Telegram provides extensive support for bot development, including official documentation and built-in tools that simplify the creation and testing of conversational agents directly within the messaging app.

While other platforms, such as WhatsApp, were also considered for integration, their use presented several limitations — including restricted community support, scarce technical documentation, and higher operational costs associated with message delivery. For these reasons, Telegram was selected as the primary communication platform for LEIA.

Telegram’s API enables the configuration of webhooks to receive real-time updates from user interactions, supporting text, image, and audio messages in a straightforward and structured way. This design facilitates the retrieval, processing, and classification of various message types. Additionally, the API provides simple methods for sending messages, files, and media back to users, which is essential for use cases such as delivering analytical plots, data tables, and system notifications.

4.6 Modularity

One of the main design principles during the development of ESIT was its modular architecture. The system was structured to allow seamless updates to the underlying LLM and straightforward integration of new tools with minimal effort. This modularity extended to its internal components as well. For instance, the Retrieval-Augmented Generation (RAG) module was designed to be data-agnostic — capable of processing and indexing any type of information, making it readily available for the agent to query. Similarly, the modeling tool was built to easily adapt to different energy system model specifications, requiring only compatibility with CESM, the solver originally integrated with ESIT.

This design choice proved highly effective during the development of LEIA. Beyond general organizational and workflow improvements, only minor adjustments were required to adapt the chatbot layer. Specifically, prompt engineering strategies were refined to accommodate the transition from the Llama model to GPT OSS, which introduced significant differences in input/output formatting and conversational control. Apart from

that, most components — including tools, graph definitions, and configuration mechanisms — were directly reused without modification.

LEIA’s modularity has been further enhanced by the abstraction introduced through the integration with Telegram, replacing the custom-built graphical interface. This shift eliminated the need to manage message delivery and visualization internally, allowing the focus to remain on response generation and interaction logic. As a result, the system can now evolve with greater flexibility: without altering the core model, it is possible to modify prompts, add or remove tools, and rapidly deploy a new chatbot configuration suited to different use cases.

4.7 Pricing

All third-party tools used in this project offer, to some extent, free usage tiers that were sufficiently powerful for our local use case. These free options allowed us to run the entire pipeline efficiently, without performance degradation, while still obtaining excellent results. Nevertheless, it is important to highlight the estimated operational costs that would be incurred if these free tiers were not available, in order to provide a realistic projection of LEIA’s potential running expenses.

4.7.1 Groq

Groq provides a generous free plan, which proved adequate for most of our development and testing needs. However, during the implementation phase, we observed occasional latency and errors due to token limit rate excess due to limitations applied to free-tier users. After validating most of the use cases and thoroughly testing the selected models, we migrated to the Developer’s Plan, with a pay-as-you-go strategy and higher TPM and RPM limits than the free tier. The current usage limits of the free plan and the developer’s plan are presented in Table 3.

Regarding pricing, the applicable rates for the models relevant to this work are shown in Table 4. Based on our usage in August 2024, when the system was being developed with the Llama 3.1 70B model, approximately 4 million input tokens and 500 thousand output tokens were processed, across a total of 4,642 requests. This volume would correspond to an estimated cost of around \$2.76 — a relatively low expense considering the intensity of usage throughout that period.

Assuming the same usage pattern with our current GPT OSS implementation, the total estimated cost would be even lower, at approximately \$0.90, while achieving superior performance and output quality.

4.7.2 LlamaCloud

LlamaCloud also provides a reasonable free plan that allows users to perform basic document parsing through its API. Free-tier users receive 10,000 credits per month, with each page processing costing between 1 credit (for non-AI parsing) and 90 credits (for more advanced use cases).

Given that ESIT’s data source consisted of only 18 pages, and assuming the recommended cost-efficient configuration of 3 credits per page, the total consumption would amount to 54 credits. With a pricing rate of \$1 per 1,000 credits, and considering that parsing was a one-time operation for our use case, the cost would be negligible in practical terms.

4.7.3 Qdrant

Qdrant offers a free-tier cluster that includes 1 GB of storage and limited processing capacity. Despite these constraints, the plan already provides essential production-grade features such as automatic scaling, logging, backups, and support for multiple users.

Since the amount of data stored in our cluster is relatively small, the free plan is expected to be sufficient for ESIT over an extended period. Nevertheless, if scaling becomes necessary, paid clusters are available starting at \$0.014 per hour (approximately \$10 per month), offering a cost-effective path to expand storage and performance capabilities when required.

4.7.4 Tavily

Among the four services used, Tavily presents the most restrictive free plan, offering only 1,000 API calls per month. During testing, this limit was reached only once — specifically while evaluating the web search functionality, which triggered multiple sequential API requests.

Given the current operational demands of the project, the free plan remains adequate. However, if the system’s scope expands, Tavily also offers paid plans ranging from 4,000 API calls per month for \$30 to 100,000 API calls per month for \$500.

Should the project require large-scale deployment in the future, Tavily would

Model	1M input tokens	1M output tokens
Llama 3.1 8B Instant	\$0.05	\$0.08
Llama 3.3 70B Versatile	\$0.59	\$0.79
GPT OSS 20B	\$0.075	\$0.30
GPT OSS 120B	\$0.15	\$0.60

Table 4 – Current usage prices for Groq’s models

likely represent the most significant share of operational costs, becoming a key point for optimization or alternative integration strategies.

5 Experiments

Developing experiments for a tool like LEIA is a challenging task. Since the input can be any natural language query and the LLM autonomously determines whether to use any of the tools and available data, the amount of free variables ends up becoming pretty big. The way we found to test the expected behavior of LEIA and verify if it matches the actual behavior was to define some use cases and check if everything happened as expected in the workflow execution.

From that we could then create a score of expected vs executed actions, as well as a more specific score for the data access cases, verifying for example if the expected data was retrieved, if the correct graphs were plotted and if the provided insights actually make sense based on the provided data.

5.1 Defining the test cases

We will start by defining some simple use cases that fall on the category of simple conversations and usage of the simple tools:

- **Basic social interactions:** Check if simple interactions such as "Hi", "How are you?", "Who are you?" and "Thanks!" lead to a direct answer from the tool;
- **Simple web searches:** Check if the tool is able to execute simple researches, such as "How is the weather today?", "What is the current conversion rate from Euro to Dollar?" or "How many medals did Germany won in the 2024 Olympic Games?";
- **Direct calculations:** Check if the tool is able to identify when the user asks about a mathematical problem and whether it is capable of solving it;
- **Complex calculations:** Check if the tool can reason to get external variables from other sources, such as the internet, to execute calculations, and also if it's able to execute multiple step calculations.

After executing the simpler tests, we can proceed to test the data retrieval and data plotting tools:

- **Consult basic usage data:** Check if the tool is able to identify when the user asks about simple aspects of the energy usage, such as total usage in a period, highest consumer, and so on;

- **Consult complex usage data:** Check the tool’s capability of reasoning over bigger chunks of data, and more complex concepts, such as outliers, peaks, and so on;
- **Plot specific data on demand:** Check if the tool can correctly identify the plots requested by a user, as well as showing them;
- **Check for maintenance history:** Check if the tool is able to consult the maintenance history of the laboratory and manipulate it.

Finally, we can also test the multi-modality capabilities of the tool, by testing if it can understand media other than text.

- **Audio understanding:** Check if the tool is able to understand and correctly answer audio messages.
- **Image understanding:** Check if the tool is able to convert image messages to explanations it can use to answer the user.

We can now present a table with each proposed user input for the two categories and the expected actions to be taken, as well as the expected results:

ID	Prompt	Expected results
1	"Hi!"	A simple salutation, potentially with the tool presenting itself.
2	"How are you?"	A simple answer plus a potential presentation of itself.
3	"Who are you?"	The tool should present itself as LEIA and tell the user what it can do.
4	"Thanks!" (after another prompt)	The tool should say something on the line of "You're welcome" and say that it's ready to help on similar matters.
5	"How is the weather in Berlin today?"	The tool should search the web for the information on the weather and answer the user with at least the temperature for the day.
6	"What is the current conversion rate from Euro to Dollar?"	The tool should be able to search for this specific information and return a correct value.
7	"How many medals did Germany won in the 2024 Olympic Games?"	The tool should be capable of searching a more generic question and finding the correct information to show the user (the expected is the total number of medals, not a specific type)
8	"How much is the square root of 217,4 plus 341,2?"	The tool should be build the correct equation to solve this problem and return the correct value.

9	"How much is the current age of Olaf Scholz to the power of 3,4?"	The tool should search for the current age of Olaf Scholz and then use the calculator tool to calculate the power.
10	"What is the age difference between Angelina Jolie and Brad Pitt to the power of 4,7?"	The tool should be able to calculate both parts of the problem after also searching the information online.
11	"What was the total usage of energy last month?"	The tool should answer with the total usage in kWh for the selected period.
12	"What was the highest consumer this week?"	The tool should correctly search for the highest usage and inform it to the user.
13	"Was there any usage outlier this month?"	The tool should correctly use the query tool to search for the outliers and inform them to the user.
14	"How much was the peak usage last week?"	The tool should answer with the peak value of energy consumption.
15	"Show me the total usage of energy by device last week"	The tool should plot only the requested data.
16	"When was the last time the AC was fixed?"	The tool should find the correct information and tell the date to the user.
17	"I changed the router to a new one today, can you register it?"	The tool should ask for the necessary details, and finally register the new maintenance.
18	"What can you do for me?" via audio	The tool should correctly understand the audio message and explain its capabilities.
19	"What is the power of this device?" with image of specific device	The tool should try to find the device on the internet, and inform the correct power.

Table 5 – Set of test cases developed to assess the general performance of LEIA

5.2 Test case results

In the next three tables we have the consolidated results of the tests executed. The two first tables will set the evaluation as a percentile score based on the following aspects: (1) the correct tool was used; (2) the tool responded exactly the asked question, with no unnecessary addends; (3) the information returned was correct. Additionally, we added a column stating the time taken from the moment the question was asked to the moment the user got the answer. The results the simple use cases can be seen in Table 6.

Second, we'll check for the more complex use cases results, using the same percentile score idea. The execution time is also added to the results, that can be verified in Table 7.

Finally, we'll also verify the multi-modality use case results. The scoring elements for this one are: (1) the tool was able to process the input; (2) the content of the input is correctly

ID	(1)	(2)	(3)	Score	Execution time
1	✓	✓	✓	100%	2.87 s
2	✓	✓	✓	100%	2.54 s
3	✓	✓	✓	100%	2.94 s
4	✓	✓	✓	100%	2.34 s
5	✓	✓	✓	100%	7.73 s
6	✓	✓	✓	100%	6.46 s
7	✓	✓	✓	100%	15.61 s
8	✓	✓	✓	100%	3.38 s
9	✓	✓	✓	100%	2.27 s
10	✓	✓	✓	100%	6.84 s

Table 6 – Experiment results for the simple use cases

ID	(1)	(2)	(3)	Score	Execution time
11	✓	✓	✓	66%	6.93 s
12	✓	✓	✓	100%	15.24 s
13	✓	✓	✓	100%	4.69 s
14	✓	✓	✓	100%	3.76 s
15	✓	✓	✓	100%	14.42 s
16	✓	✓	✓	100%	4.67 s
17	✓	✓	✓	100%	5.37 s

Table 7 – Experiment results for the complex use cases

extracted; (3) the LLM is able to correctly respond the user with the processed input. The results are available in Table 8.

ID	(1)	(2)	(3)	Score	Execution time
18	✓	✓	✓	100%	5.18 s
19	✓	✓	✗	66%	5.64 s

Table 8 – Experiment results for the multi-modality use cases

5.3 Consistency tests

Another important point to consider when developing a LLM-based tool is that language models are by nature non-deterministic. For the same input the answer will probably never be the same. When developing a multi-tool LLM agent we need to at least make sure that the used tools and selected input will keep the same through different iterations of the same input. However, there are some steps in which we can't have that type of control, for example, the web search feature may use different queries for the same input, leading to different results.

To assess the impacts of this specific situation, we have developed a separate set of tests aimed completely at understanding where this type of situation can happen, and how much it can influence on the desired result. In Table 9 we have the selected inputs to be tested, each of them will be executed three times, and we'll then analyze both the actions taken by the agent, the intermediate queries used by the model to get to the final result.

ID	Prompt	Description
1	"Show me the usage data for last month"	Requests a really open scenario, where the tool can end up checking different usage aspects in each run.
2	"How is the weather in Berlin right now?"	Requests a question that can be interpreted in different ways by the LLM, either the current weather or the typical weather for now. The answer will depend on the sources it find.
3	"Who made the last maintenance?"	The tool can end up asking for more details or go directly for the last maintenance.
4	"How can I reduce the energy usage?"	The tool may end up searching the internet for energy usage reduction techniques, which may lead to different answers each time.

Table 9 – Set of test cases developed to assess consistency of the workflow execution for the same input

To execute these tests, we'll use the following method: the LLM memory will be reset for every run to avoid the context bias; the evaluation will take in account the intermediate queries used and the final result of the bot; the final answer will be considered identical as long as the presented key information is the same, variations in the answer text won't be accounted as inconsistencies, given that this is the normal behavior of a LLM. In Table 10 we can check how every test case performed. The criteria are: (1) the data requested/modified/searched was consistent over the iterations; (2) the final result was consistent over iterations; (3) the required time was similar for every iteration; (4) the data was always the same for the same request.

ID	(1)	(2)	(3)	(4)	Score
1	✓	✓	✓	✓	100%
2	✓	✗	✓	✗	50%
3	✓	✓	✓	✓	100%
4	✓	✓	✓	✗	75%

Table 10 – Experiment results for the consistency tests

5.4 Test Results

The test results demonstrated a substantial improvement over ESIT. In its original evaluation, using similar tests and metrics, ESIT achieved an average score of approximately 82% and a mean execution time of around 44 seconds. Thanks to LEIA’s simplified state management, straightforward execution loop, and smoother UI integration, these numbers improved significantly: LEIA achieved an average score of roughly 96% and a mean execution time of just 6.2 seconds.

LEIA not only improved the quality of the generated results but also captured the user’s intent more precisely. The only notable limitation appeared in generic queries about images. Since the prompt is strongly oriented toward energy monitoring, asking about the energy consumption of arbitrary devices sometimes led the model to general explanations about laboratory consumption instead of addressing the specific device.

Regarding consistency tests, the results followed a pattern similar to ESIT. Although tool usage remained consistent and the overall direction of each response aligned with expectations, the emphasis varied between iterations—sometimes focusing more on the provided data, other times on tips and insights. The least reliable use case was the generic web search, whose results varied significantly across identical queries. This highlights that non-determinism does not stem solely from the model, but also from the tools it depends on.

Overall, all proposed use cases were successfully validated. In every test, the model provided useful information, relevant insights, or actionable recommendations. The tools were well-tailored for LLM integration, which contributed substantially to the performance observed throughout the evaluation.

5.5 Tool Performance

As discussed in Section 5.4, both performance and output quality improved considerably compared to the previous ESIT iteration. Execution times, which previously ranged from 2 seconds to 4 minutes, were reduced to a far more acceptable window of 2 to 15 seconds. The few cases that reached the upper bound were those involving data gathering and plotting workflows, while the remaining use cases typically executed in under 6 seconds. This results in a much smoother and more pleasant user experience, as the waiting time for responses is dramatically reduced.

During testing, the overall interaction with the agent was highly positive. It consistently understood what the user intended to view, verify, or search for, and it handled these requests accurately in most cases. This yielded valuable insights into laboratory energy usage and reinforced the usefulness of the tool.

From a cost perspective, LEIA benefits from well-designed tools and broad use cases, which allowed us to avoid overly large prompts or complex prompt engineering strategies. As a consequence, inference times are shorter, token usage is lower, and fewer requests are needed to

obtain high-quality results. This led to substantially lower operational costs when running on Groq. After switching to the developer tier, the cost per message averaged below \$0.001, which is remarkably low given the quality and sophistication of the agent.

5.6 Observed problems

Most of the issues observed in the test scenarios presented in Table 5 were related to the model’s difficulty in handling complex information-gathering inputs or in generating insights that were irrelevant to the requested data. These problems stem primarily from the inherent non-determinism of the system and the large volume of information provided at each iteration.

Beginning with the less critical cases, test case 6 exposed the system’s inability to retrieve information that is embedded or obscured within multiple layers of content. The web search queries were not precise enough to consistently retrieve the same information, causing both the data sources and the returned results to vary across executions. In test cases 9 and 10, the main issue was that the language model attempted to infer the requested ages directly rather than relying on the web search tool. Although it often retrieved correct values, failing to perform the search for less common or less relevant information led to inaccuracies. Nonetheless, the numerical calculations themselves were always correct, reflecting the model’s strong performance in mathematical reasoning even without the aid of a calculator tool.

For test case 19, the model struggled to determine how to use the generated image description. It attempted to connect information extracted from the image with the user’s question but defaulted to insights based on laboratory energy data—its domain of expertise—instead of, for example, searching online for the specific information needed.

All of these shortcomings have the potential to produce misleading analyses for the user, since the model relies entirely on the data supplied to it. LEIA’s development process is therefore a continuous balancing act: designing deterministic queries that reliably provide the necessary information to the agent while avoiding over-constraining the system in ways that limit its natural-language reasoning capabilities. The tool must remain flexible enough to support broad and general-purpose usage while maintaining sufficient determinism in its operations—an ongoing effort to avoid compromising either side too heavily.

6 Results

We have now already presented all details of LEIA functions, how do they work and how well the tool performs many different types of user requests. With all of that information, we can finally reach some conclusions about the possibilities allowed by it and the identified problems and flaws found during the tests and experiments.

6.1 Impacts of non-determinism

Undoubtedly the biggest problem we faced during the development of LEIA was the non-determinism inherent to Large Language Models. A study that evaluates the non-determinism in code generation ([OUYANG et al., 2023](#)) has shown the high degree of variation in code snippets generated using the same prompt, many times leading to complete different test results. These results are also corroborated by ([ASTEKIN; HORT; MOONEN, 2024](#)), which also show how the non-determinism influences the parsing of system logs. Both studies reached the conclusion that even reducing the model temperature to 0, a parameter that dictates the predictability of the LLM, the outputs generated varies in levels that can generate problems for applications that rely on standardized outputs.

Considering this, the consideration of non-determinism in LLM evaluations is set to play a big role in future assessments of LLM-based applications. In some cases, it's even possible for smaller models to have better reliability than bigger ones ([SONG et al., 2024](#)), causing a situation where the model selection will end up playing a huge role in the final results of the applications. Another way of surpassing this limitations without giving up on the larger models capabilities is based on using guardrails ([Ayyamperumal; GE, 2024](#)), such as specific tailored system prompts and usage of RAG to allow the gathering of correct information always, as a protection layer to avoid the problems caused by the underlying non-determinism of LLMs.

This was in fact the solution found to better control and limit the outputs generated by LEIA. All of the internal tools are limited to output specific terms organized in JSON objects that can be later parsed and correctly used by the main agent. Even the output formatter, responsible for generating the final answer to the user, uses JSON output parsing to ensure the correct format. However, in the end, even using all this protection layers, this characteristic of language models still result in most of the problems encountered during the development and evaluation of LEIA.

6.2 Trial and error

One of the biggest drawbacks of designing a tool based on LLMs is that some layers of control that we could have with a normal, deterministic tool are simply not reachable. The

process of developing the agent uses a lot of the idea of fine tuning the prompt so it reaches the desired behavior.

Integrating tools also requires the same level of prompt engineering, since most of the tools usefulness to the agent will be determined by how well the tool description is presented to the LLM model.

Through a long process of writing and re-writing prompts and tool descriptions, and refining the agent workflow, as well as the Telegram integration, we were able to reach a functioning tool that is capable of executing all of the main tasks required of a energy consumption monitoring tool, as well as some other less specific tasks, such as searching general information online.

This represented most of the development time, and even after spending lots of attention to the possible scenarios that could be presented to LEIA, we weren't able of fully covering everything. Some use cases still don't work 100% as expected, and may require new re-writing iterations to reach the desired level of quality.

6.3 Positive results

Putting the presented problems aside, the performance of LEIA actually ended up being pretty impressive. The final version of the agent is able to understand different types of input messages, detect the user's intent, detect the correct data aspects required for a query most of the time and even select the correct plot types for each data request.

This in itself presents a great opportunity for better observability of energy usage, in a way that anyone working in the laboratory could have a virtual partner to help understand the real impacts of adding new devices, modifying the current organization and even monitoring the laboratory usage in real time.

Another interesting point to note is that, since LEIA offers a good level of modularity, it could be enhanced or even completely repurposed by simply developing new tools and modifying the base prompt of the agent. This represents one of the greatest benefits of using a model such as GPT OSS with tooling capabilities instead of Llama. ESIT required quite a bit of work to integrate new tools and use cases since it had a complex state and flow management, while the improvements introduced by LEIA's usage of GPT OSS make this process way simpler, since the model is able to select the correct tools by itself.

In short, LEIA shows that it is possible to reliably use LLMs to help manage energy consumption, and, while we showed that using the limited scope of a university laboratory, the benefits LEIA provides could be replicated to much bigger environments, such as offices, factories and big stores.

7 Outlook

The core idea of using an LLM-based tool for data monitoring—energy usage in our case—offers significant potential across many fields that rely on historical data analysis and insight generation. By enabling fast and intuitive access to complex datasets, while also providing contextualized interpretations and recommendations, LLM-powered systems can substantially reduce the technical barrier for both managers and researchers. Through carefully crafted prompting strategies and by delegating to the model the ability to decide which steps to take, which parameters to inspect, and which visualizations to generate, it becomes possible to build semi-autonomous systems capable of handling much of the manual work typically associated with data processing. LEIA was developed precisely with this ambition, aiming to go beyond a simple assistant and offer meaningful analytical support.

LEIA was designed to understand a wide range of analytical requests, enabling deeper exploration of any available historical dataset. In our specific use case—simplifying the management of laboratory energy usage—this means reducing the friction in how users access information. Instead of relying on dashboards or performing manual and time-consuming analysis, the user can simply message a bot and receive clear, contextual, and complete explanations about any energy-related question.

By leveraging the LangChain library and OpenAI’s GPT-OSS models, we built a graph-based system that extracts the potential of LLMs to orchestrate tools, retrieve data, generate insights, and present visualizations in a coherent and accessible way. The evaluation of LEIA showed that the system is functional and aligned with the proposed application domain, enabling fast and intuitive interaction with the underlying database. Moreover, LEIA’s web capabilities allow it to provide up-to-date information on essentially any topic the user may inquire about, not limited to the laboratory itself.

However, reality proves itself to be always harder than what expected. Some relevant problems were found during the evaluation and testing process of the tool. Some of the biggest points of concern were: (1) the limited ability of the web search feature to get complete information about the requested topic; (2) the difficulty of using image inputs as context for other purposes; (3) the problems related to interacting with the maintenance spreadsheet; (4) and problems related to memory management. All of these situations together have the potential of causing wrong understandings about the data and in some cases even lead to wrong decisions, putting on hold the possibility of reliably using the system in critical situations.

7.1 Future work

Based on the evaluation results and the performance limitations observed during development, we identify a set of improvements that could guide future iterations.

From a performance and reliability standpoint, two changes are particularly relevant. First, hosting the LLM on a dedicated proprietary server—or alternatively migrating to a paid plan with higher limits—could improve stability and reduce restrictions imposed by third-party APIs. Second, deploying a self-hosted vector store for the RAG components would increase reliability and offer full control over file management, improving consistency in document treatment and long-term maintainability.

Compared with the previous iteration of the project (ESIT), LEIA already represents a substantial improvement. We moved from a bespoke interface with limited scalability to a Telegram-based setup that naturally supports concurrency and ease of use. We also streamlined the workflow into a single agent with multiple tools—something previously limited by the performance of Llama-based models—greatly improving efficiency, reliability, and transparency.

Nonetheless, there are still several aspects that could be refined. For the specific task of accessing historical energy-consumption data, a dedicated backend could pre-compute commonly used aggregations, reducing overhead for the LLM and improving consistency. Additionally, a parallel notifications system—separate from the bot and capable of reaching users even during internet outages—would significantly increase the robustness of the solution in safety-critical environments. Finally, while restricting conversation context to the last ten messages works reasonably well today, implementing a long-term memory architecture would improve continuity and support more complex multi-step interactions.

7.2 Conclusion

The development and evaluation of LEIA demonstrate the strong potential of LLM-based tools for energy-consumption monitoring. The system shows that, with the current generation of open-weight language models, it is possible to build a reliable and coherent assistant capable of simplifying access to technical data and supporting decision-making for laboratory managers.

More importantly, LEIA illustrates how conversational interfaces can transform the way users interact with data systems. By turning complex queries into natural language exchanges, LLM-driven tools can eliminate many of the traditional barriers associated with dashboards, manual analysis, or specialized software.

Beyond our specific use case, the approach taken in LEIA opens possibilities for numerous other domains where data accessibility and interpretability are major challenges. It is easy to imagine similar systems supporting environmental monitoring, industrial processes, building automation, or even educational laboratories—where interacting with historical data becomes as simple as holding a conversation with an intelligent agent.

Overall, while limitations remain and further work is needed to reach full reliability, the results of this project highlight a promising direction for future human-AI collaboration in data-driven environments.

Bibliography

- ACHIAM, J. et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. Citado na página 15.
- AFZAL, U. et al. Investigating cognitive load in energy network control rooms: Recommendations for future designs. *Frontiers in psychology*, Frontiers Media SA, v. 13, p. 812677, 2022. Citado na página 14.
- AGARWAL, S. et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025. Citado 4 vezes nas páginas 8, 20, 21, and 37.
- AI, M. *Llama 3.1 / Model Cards and Prompt Formats*. 2024. <https://llama.meta.com/docs/model-cards-and-prompt-formats/llama3_1/> [Accessed: 26.08.2024]. Citado na página 40.
- AI, M. *Llama 3.1 Requirements*. 2024. <<https://llamaimodel.com/requirements/>> [Accessed: 24.08.2024]. Citado na página 19.
- ANTHROPIC. The claude 3 model family: Opus, sonnet, haiku. In: . [s.n.], 2023. Disponível em: <<https://api.semanticscholar.org/CorpusID:268232499>>. Citado na página 15.
- ASTEKIN, M.; HORT, M.; MOONEN, L. An exploratory study on how non-determinism in large language models affects log parsing. In: *Proceedings of the ACM/IEEE 2nd International Workshop on Interpretability, Robustness, and Benchmarking in Neural Software Engineering*. New York, NY, USA: Association for Computing Machinery, 2024. (InteNSE '24), p. 13–18. ISBN 9798400705649. Disponível em: <<https://doi.org/10.1145/3643661.3643952>>. Citado na página 56.
- AYYAMPERUMAL, S. G.; GE, L. *Current state of LLM Risks and AI Guardrails*. 2024. Disponível em: <<https://arxiv.org/abs/2406.12934>>. Citado na página 56.
- BARBOSA, J.; RIPP, C.; STEINKE, F. Accessible modeling of the german energy transition: An open, compact, and validated model. *Energies*, v. 14, n. 23, 2021. ISSN 1996-1073. Disponível em: <<https://www.mdpi.com/1996-1073/14/23/8084>>. Citado 2 vezes nas páginas 17 and 25.
- BARBOSA, S. H. J. *CESM*. [S.l.]: GitHub, 2023. <<https://github.com/EINS-TUDa/CESM>>. Citado na página 25.
- BOREK, C. Comparative evaluation of llm-based approaches to chatbot creation. 2024. Citado na página 22.
- BROWN, T. B. et al. Language models are few-shot learners. *ArXiv*, abs/2005.14165, 2020. Disponível em: <<https://api.semanticscholar.org/CorpusID:218971783>>. Citado na página 27.
- CHEN, J.-W.; ZHANG, J. Comparing text-based and graphic user interfaces for novice and expert users. In: AMERICAN MEDICAL INFORMATICS ASSOCIATION. *AMIA annual symposium proceedings*. [S.l.], 2007. v. 2007, p. 125. Citado na página 22.
- DELFIANTI, R. et al. Internet of things (iot) based electrical power monitoring system for solar power plants using telegram application. *Vokasi Unesa Bulletin of Engineering, Technology and Applied Science*, v. 2, n. 3, p. 428–443, 2025. Citado na página 15.
- DUBEY, A. et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. Citado 4 vezes nas páginas 8, 15, 18, and 19.

- GROQ. *Groq Supported Models*. 2024. <<https://console.groq.com/docs/models>> [Accessed: 25.08.2024]. Citado na página 24.
- GROQ. *GroqCloud*. 2024. <<https://groq.com/groqcloud/>> [Accessed: 25.08.2024]. Citado na página 24.
- GROQ. Groqthoughts: What is a language processing unit? 2024. Citado na página 24.
- GUPTA, A. et al. Llm task interference: Impact of task-switch in conversational history. In: *ICML 2024 Workshop on Foundation Models in the Wild*. [S.l.: s.n.], 2024. Citado na página 44.
- HATALIS, K. et al. Memory matters: The need to improve long-term memory in llm-agents. In: *Proceedings of the AAAI Symposium Series*. [S.l.: s.n.], 2023. v. 2, n. 1, p. 277–280. Citado na página 44.
- HENDRIX, G. G. et al. Developing a natural language interface to complex data. *ACM Transactions on Database Systems (TODS)*, ACM New York, NY, USA, v. 3, n. 2, p. 105–147, 1978. Citado na página 22.
- JAIN, S. M. Hugging face. In: *Introduction to transformers for NLP: With the hugging face library and models to solve problems*. [S.l.]: Springer, 2022. p. 51–67. Citado na página 15.
- JEONG, J. et al. Current research and future directions for off-site construction through langchain with a large language model. *Buildings*, MDPI, v. 14, n. 8, p. 2374, 2024. Citado na página 24.
- KG, B. A. G. . C. *The complete energy measurement chain: from the sensor to the cloud*. Verl, Germany, 2024. Flyer. Retrieved from <<https://www.beckhoff.com/media/downloads/information-media/flyer-energy-measurement-2.pdf>>. Disponível em: <<https://www.beckhoff.com/media/downloads/information-media/flyer-energy-measurement-2.pdf>>. Citado na página 14.
- LI, H.; APPLEBY, G.; SUH, A. Linkq: An llm-assisted visual interface for knowledge graph question-answering. *arXiv preprint arXiv:2406.06621*, 2024. Citado na página 22.
- LI, Y. A practical survey on zero-shot prompt design for in-context learning. In: *Proceedings of the Conference Recent Advances in Natural Language Processing - Large Language Models for Natural Language Processings*. INCOMA Ltd., Shoumen, BULGARIA, 2023. p. 641–647. Disponível em: <http://dx.doi.org/10.26615/978-954-452-092-2_069>. Citado na página 27.
- LIU, J. *LlamaIndex*. 2022. Disponível em: <https://github.com/jerryliu/llama_index>. Citado na página 26.
- LIU, J. *LlamaParse*. 2022. Disponível em: <https://docs.cloud.llamaindex.ai/llamaparse/getting_started>. Citado na página 26.
- MAHARANA, A. et al. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024. Citado na página 44.
- MAVROUDIS, V. *LangChain*. London, United Kingdom, 2024. Preprint. Citation ID: 5977565. Disponível em: <<https://www.turing.ac.uk/sites/default/files/2024-11/langchain.pdf>>. Citado na página 15.
- NAVEED, H. et al. A comprehensive overview of large language models. *arXiv preprint arXiv:2307.06435*, 2023. Citado na página 18.

- NREL, N. R. E. L. *Laboratories for the 21st Century: An Introduction to Low-Energy Design (Revised)*. [S.l.], 2008. Disponível em: <<https://www.osti.gov/biblio/907998>>. Citado na página 14.
- OPPENLAENDER, J.; LINDER, R.; SILVENNOINEN, J. *Prompting AI Art: An Investigation into the Creative Skill of Prompt Engineering*. 2024. Disponível em: <<https://arxiv.org/abs/2303.13534>>. Citado na página 26.
- OUYANG, S. et al. *LLM is Like a Box of Chocolates: the Non-determinism of ChatGPT in Code Generation*. 2023. Disponível em: <<https://arxiv.org/abs/2308.02828>>. Citado na página 56.
- PANDYA, K.; HOLIA, M. Automating customer service using langchain: Building custom open-source gpt chatbot for organizations. *arXiv preprint arXiv:2310.05421*, 2023. Citado na página 24.
- QDRANT. *Qdrant Vector Database*. 2024. <<https://qdrant.tech/qdrant-vector-database/>> [Accessed: 25.08.2024]. Citado na página 26.
- RIVADENEIRA, W.; BEDERSON, B. B. A study of search result clustering interfaces: Comparing textual and zoomable user interfaces. *University of Maryland HCIL Technical Report HCIL-2003*, Citeseer, v. 36, 2003. Citado na página 14.
- SCAO, T. L. et al. Bloom: A 176b-parameter open-access multilingual language model. 2023. Citado na página 15.
- SHANAHAN, M. Talking about large language models. *Communications of the ACM*, ACM New York, NY, USA, v. 67, n. 2, p. 68–79, 2024. Citado na página 14.
- SHEN, L. et al. Towards natural language interfaces for data visualization: A survey. *IEEE transactions on visualization and computer graphics*, IEEE, v. 29, n. 6, p. 3121–3144, 2022. Citado na página 22.
- SINGH, A. et al. Revolutionizing mental health care through langchain: A journey with a large language model. In: IEEE. *2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)*. [S.l.], 2024. p. 0073–0078. Citado na página 24.
- SONG, Y. et al. *The Good, The Bad, and The Greedy: Evaluation of LLMs Should Not Ignore Non-Determinism*. 2024. Disponível em: <<https://arxiv.org/abs/2407.10457>>. Citado na página 56.
- TAVILY. *Tavily*. 2024. <<https://tavily.com/>> [Accessed: 25.08.2024]. Citado na página 25.
- TEAM, G. et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023. Citado na página 15.
- Telegram Messenger LLP. *Telegram Applications*. 2025. <<https://telegram.org/apps>>. Accessed: 2025-11-09. Citado na página 45.
- TOPSAKAL, O.; AKINCI, T. C. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In: *International Conference on Applied Engineering and Natural Sciences*. [S.l.: s.n.], 2023. v. 1, n. 1, p. 1050–1056. Citado na página 24.
- UMAR, F. *Enhancing Document Accessibility and User Interaction through Large Language Model: A Comparative Study for Educational Content: A Comparative Analysis of LLM and Traditional Site Search*. 2024. Citado na página 22.

VERA-PIAZZINI, O.; SCARPA, M.; PERON, F. Building energy simulation and monitoring: a review of graphical data representation. *Energies*, MDPI, v. 16, n. 1, p. 390, 2022. Citado na página 14.

WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. In: KOYEJO, S. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2022. v. 35, p. 24824–24837. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf>. Citado na página 27.

WEI, J. et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. Disponível em: <<https://arxiv.org/abs/2201.11903>>. Citado na página 33.

WOODSIDE, T.; TONER, H. *Multimodality, Tool Use, and Autonomous Agents: Large Language Models Explained, Part 3*. 2024. <<https://cset.georgetown.edu/article/multimodality-tool-use-and-autonomous-agents/>> [Accessed: 25.08.2024]. Citado na página 34.

YAO, S. et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. Disponível em: <<https://arxiv.org/abs/2210.03629>>. Citado na página 27.

YE, X.; DURRETT, G. The unreliability of explanations in few-shot prompting for textual reasoning. *Advances in neural information processing systems*, v. 35, p. 30378–30392, 2022. Citado na página 27.

ZAMFIRESCU-PEREIRA, J. et al. Why johnny can't prompt: How non-ai experts try (and fail) to design llm prompts. In: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2023. ISBN 9781450394215. Disponível em: <<https://doi.org/10.1145/3544548.3581388>>. Citado na página 26.

ZHANG, K. et al. Memory-augmented llm personalization with short-and long-term memory coordination. *arXiv preprint arXiv:2309.11696*, 2023. Citado na página 44.

APPENDIX A – Multitool agent prompt

Listing A.1 – Prompt for the multitool agent

SYSTEM

You are a helpful AI assistant, with access to some specific tools.
Use the provided tools to progress towards answering the question.
If you are unable to fully answer, that's OK, execute what you can to make progress.

You have access to the following tools, described in JSON with the description of each parameter, the required parameters for each function, and the possible options for some parameters (in these cases, use only the available options):

```
{tool_descriptions}
```

The way you use the tools is by specifying a json blob.

Specifically, this json should have a 'action' key (with the name of the tool to use) and a 'action_input' key (with the input to the tool going here).

The only values that should be in the "action" field are: {tool_names}
The \$JSON_BLOB should only contain a SINGLE action, do NOT return a list of multiple actions. Here is an example of a valid \$JSON_BLOB:

```
'''
{{
  "action": $TOOL_NAME,
  "action_input": $INPUT
}}
'''
```

ALWAYS use the following format:

```
Question: the input question you must answer
Thought: you should always think about what to do
Action:
'''
$JSON_BLOB
'''

Observation: the result of the action
... (this Thought/Action/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question
```

Begin! Reminder to always use the exact characters 'Final Answer' when responding.

HUMAN

```
{input}
{agent_scratchpad}
```